

Database Management System

Module 1

INTRODUCTION TO DATABASES



Introduction

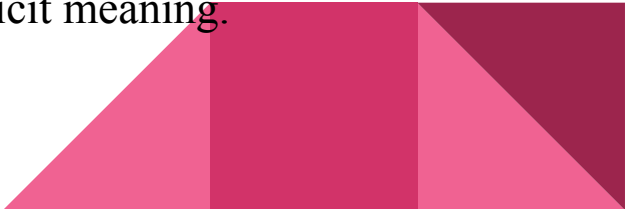
Database systems have become an essential component of life in modern society, in that many frequently occurring events trigger the accessing of at least one database

Traditional vs. more recent applications of databases

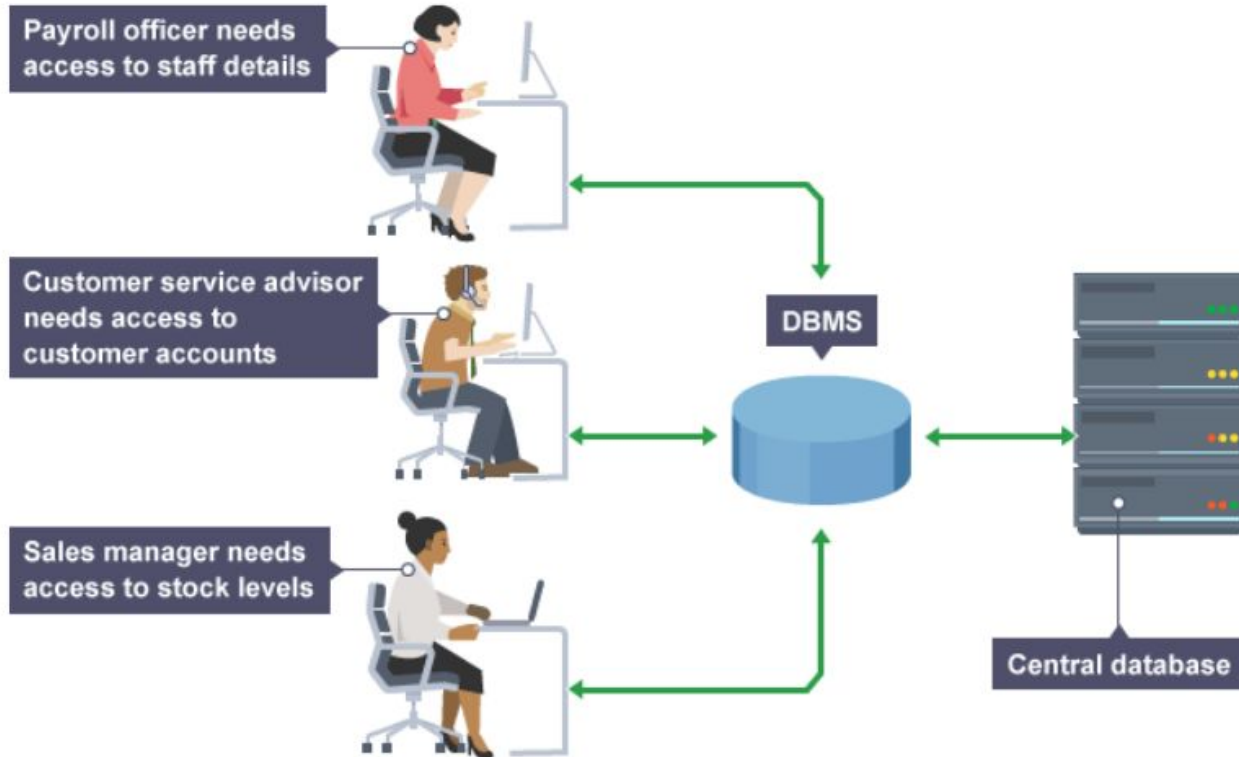
- "**Traditional**" ones use rigidly-structured **textual** and **numeric data**.
- Recent advances have led to the application of database technology to a wider class of data. Examples include **multimedia databases** (involving pictures, video clips, and sound messages) and **geographic databases** (involving maps, satellite images).

A **database** is a collection of related data.

A **data** mean known facts that can be recorded and that have implicit meaning.



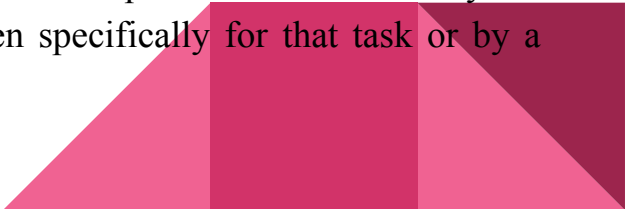
Employees are accessing Data through DBMS



A **database** has the following **implicit properties**:

1. A database represents some aspect of the real world, sometimes called the **miniworld** or the **universe of discourse (UoD)**. Changes to the miniworld are reflected in the database.
2. A database is a **logically coherent** collection of data with some inherent meaning. A random assortment of data cannot correctly be referred to as a database.
3. A database is **designed, built, and populated with data** for a specific purpose. It has an intended group of users and some preconceived applications in which these users are interested.

Other properties:

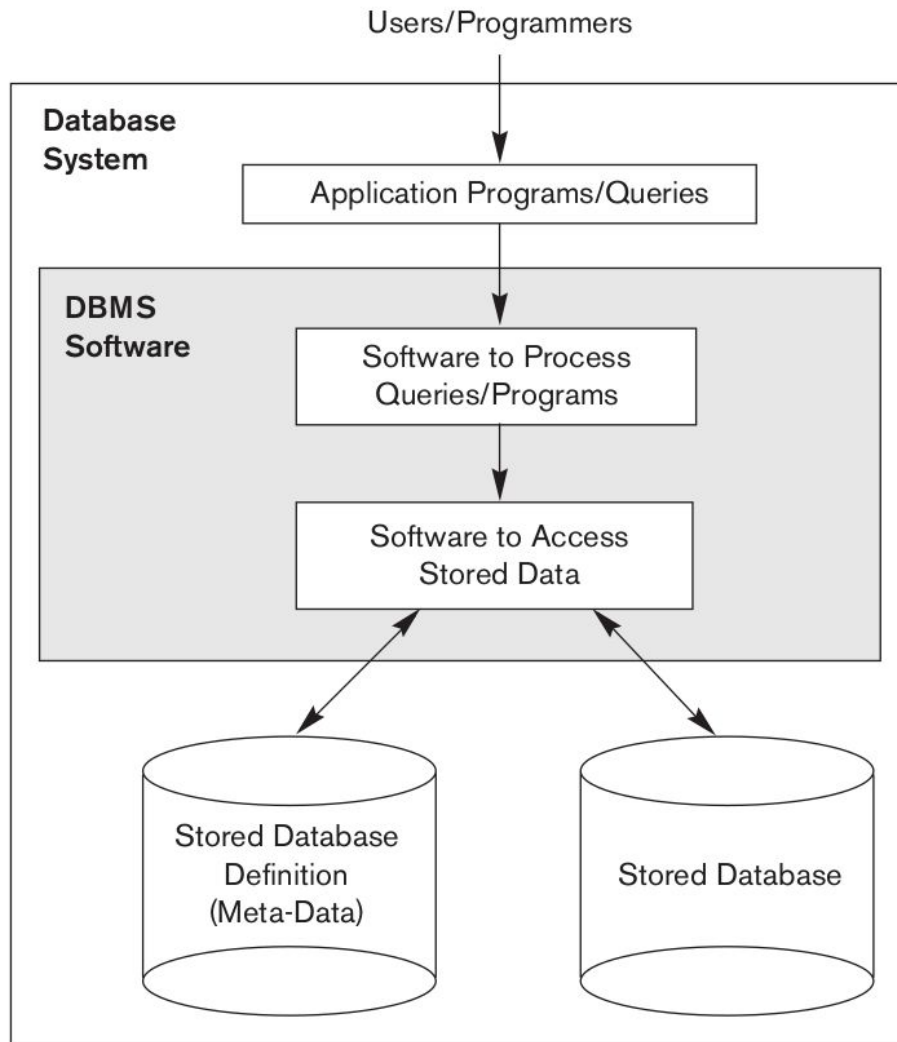
1. A database can be of any **size and complexity**. For example, the list of names and addresses referred to earlier may consist of only a few hundred records, each with a simple structure. An example of a large commercial database is Amazon.com.
 2. A database may be generated and maintained **manually or it may be computerized**. For example, a library card catalog is a database that may be created and maintained manually. A computerized database may be created and maintained either by a group of application programs written specifically for that task or by a database management system.
- 

*The DBMS is a **general-purpose software system** that facilitates the processes of **defining, constructing, manipulating, and sharing databases** among various users and applications.*



- **Defining** a database involves specifying the data types, structures, and constraints of the data to be stored in the database. The database definition or descriptive information is also stored by the DBMS in the form of a database catalog or dictionary; it is called **meta-data**.
- **Constructing** the database is the process of storing the data on some storage medium that is controlled by the DBMS.
- **Manipulating** a database includes functions such as querying the database to retrieve specific data, updating the database to reflect changes in the miniworld, and generating reports from the data.
- **Sharing** a database allows multiple users and programs to access the database simultaneously
- An **application program** accesses the database by sending queries or requests for data to the DBMS.
- A **query** typically causes some data to be retrieved; a **transaction** may cause some data to be read and some data to be written into the database.
- **Protection** includes system protection against hardware or software malfunction (or crashes) and security protection against unauthorized or malicious access.
- A typical large database may have a life cycle of many years, so the DBMS must be able to **maintain** the database system by allowing the system to evolve as requirements change over time.
- Database and DBMS software together a **Database system**.

A simplified database system environment



A database that stores
student and course
information.

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

- The database is organized as five files, each of which stores **data records** of the same type.
- To define this database, we must specify the structure of the records of each file by specifying the different types of **data elements** to be stored in each record.
- We must also specify a **data type** for each data element within a record.
- Most medium-size and large databases include many types of records and have many relationships among the records.



Database manipulation involves querying and updating. Examples of queries are as follows:

- Retrieve the transcript—a list of all courses and grades—of ‘Smith’
- List the names of students who took the section of the ‘Database’ course offered in fall 2008 and their grades in that section
- List the prerequisites of the ‘Database’ course

Examples of updates include the following:

- Change the class of ‘Smith’ to sophomore
- Create a new section for the ‘Database’ course for this semester
- Enter a grade of ‘A’ for ‘Smith’ in the ‘Database’ section of last semester

Of course, a query/update must be conveyed to the DBMS in a precise way (via the query language of the DBMS) in order to be processed



The Information Technology (IT) department within a company designs and maintains an information system consisting of various computers, storage systems, application software, and databases.

Design of a new application for an existing database or design of a brand new database starts off with a phase called **requirements specification and analysis**.

These requirements are documented in detail and transformed into a **conceptual design** that can be represented and manipulated using some computerized tools so that it can be easily maintained, modified, and transformed into a database implementation.

The design is then translated to a **logical design** that can be expressed in a data model implemented in a commercial DBMS. The final stage is **physical design**, during which further specifications are provided for storing and accessing the database.

The database design is implemented, populated with actual data, and continuously maintained to reflect the state of the miniworld.



Characteristics of the Database Approach

The main characteristics of the **database approach** versus the **file-processing approach** are the following:

- Self-describing nature of a database system
- Insulation between programs and data, and data abstraction
- Support of multiple views of the data
- Sharing of data and multiuser transaction processing



Self-describing nature of a database system

DB approach

- A fundamental characteristic of the database approach is that the database system contains not only the database itself but also a **complete definition or description of the database structure and constraints**.
- This definition is stored in the **DBMS catalog**, which contains information such as the structure of each file, the type and storage format of each data item, and various constraints on the data.
- The information stored in the catalog is called **meta-data**, and it describes the structure of the primary database.
- Whenever a request is made to access, say, the Name of a STUDENT record, the **DBMS software** refers to the catalog to determine the structure of the STUDENT file and the position and size of the Name data item within a STUDENT record

Traditional file processing

- In traditional file processing, data definition is typically part of the application programs themselves. Hence, these programs are constrained to work with only one specific database, whose structure is declared in the application programs.
 - In a typical file-processing application, the file structure and, in the extreme case, the exact location of Name within a STUDENT record are already coded within each program that accesses this data item
- 

An example of a database catalog for the database

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....		
....		
....		
Prerequisite_number	XXXXNNNN	PREREQUISITE

Insulation between programs and data, and data abstraction

- In **traditional file processing**, the structure of data files is embedded in the application programs, so any changes to the structure of a file may require changing all programs that access that file.
- For example, a file access program may be written in such a way that it can access only STUDENT records of the structure. If we want to add another piece of data to each STUDENT record, say the Birth_date, such a program will no longer work and must be changed
- By contrast, **DBMS** access programs do not require such changes in most cases. The structure of data files is stored in the DBMS catalog separately from the access programs. We call this property **program-data independence**.
- By contrast, in a DBMS environment, we only need to change the description of STUDENT records in the catalog to reflect the inclusion of the new data item Birth_date; no programs are changed. The next time a DBMS program refers to the catalog, the new structure of STUDENT records will be accessed and used

Data Abstraction:

A **data model** is used to hide storage details and present the users with a conceptual view of the database.

Programs refer to the data model constructs rather than data storage details



Internal storage format for a STUDENT record, based on the database catalog of Slide No. 15.

Data Item Name	Starting Position in Record	Length in Characters (bytes)
Name	1	30
Student_number	31	4
Class	35	1
Major	36	4

Support of multiple views of the data

- A database typically has many types of users, each of whom may require a different perspective or view of the database.
- A view may be a **subset** of the database or it may contain virtual data that is derived from the database files but is not explicitly stored.
- A multiuser DBMS whose users have a variety of distinct applications must provide facilities for defining multiple views.
- For example, **one user** of the database may be interested only in **accessing and printing** the transcript of each student. A **second user**, who is interested only in checking that students have taken all the **prerequisites of each course** for which the student registers, may require the view



TRANSCRIPT

Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

(a)

COURSE_PREREQUISITES

Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

(b)

Sharing of data and multiuser transaction processing

- A multiuser DBMS, as its name implies, must allow multiple users to access the database at the same time.
 - This is essential if data for multiple applications is to be integrated and maintained in a single database.
 - A fundamental role of multiuser DBMS software is to ensure that concurrent transactions operate correctly and efficiently.
 - A transaction is an executing program or process that includes one or more database accesses, such as reading or updating of database records.
 - The DBMS must enforce several **transaction properties**. The **isolation property** ensures that each transaction appears to execute in isolation from other transactions, even though hundreds of transactions may be executing concurrently. The **atomicity property** ensures that either all the database operations in a transaction are executed or none are.
- 

Database Users

These apply to "large" databases, not "personal" databases that are defined, constructed, and used by a single person via, say, Microsoft Access.

Users may be divided into

- Those who actually use and control the database content, and those who design, develop and maintain database applications (called —**Actors on the Scene**),
- Those who design and develop the DBMS software and related tools, and the computer systems operators (called —**Workers Behind the Scene**).



1. **Database Administrator (DBA):** This is the chief administrator, who oversees and manages the database system (including the data and software). Duties include authorizing users to access the database, coordinating/monitoring its use, acquiring hardware/software for upgrades, etc. In large organizations, the DBA might have a support staff.
2. **Database Designers:** They are responsible for identifying the data to be stored and for choosing an appropriate way to organize it. They also define views for different categories of users. The final design must be able to support the requirements of all the user sub-groups.
3. **End Users:** These are persons who access the database for querying, updating, and report generation. They are main reason for database existence!

Casual end users: Use database occasionally, needing different information each time; use query language to specify their requests; typically middle- or high-level managers.

Naive/Parametric end users: Typically the biggest group of users; frequently query/update the database using standard canned transactions that have been carefully programmed and tested in advance.

Sophisticated end users: Engineers, scientists, business analysts who implement their own applications to meet their complex needs.

Stand-alone users: Use "personal" databases, possibly employing a special- purpose (e.g., financial) software package. Mostly maintain personal databases using ready-to-use packaged applications.

An example is a tax program user that creates its own internal database.

Another example is maintaining an address book

4. System Analysts, Application Programmers, Software Engineers:

- System Analysts: determine needs of end users, especially naive and parametric users, and develop specifications for canned transactions that meet these needs.
- Application Programmers: Implement, test, document, and maintain programs that satisfy the specifications mentioned above.



Workers behind the Scene

Persons are typically not interested in the database content itself known as workers behind the scene, and they include the following categories:

DBMS system designers and implementers design and implement the DBMS modules and interfaces as a software package.

Tool developers design and implement tools—the software packages that facilitate database modeling and design, database system design, and improved performance. They include packages for database design, performance monitoring, natural language or graphical interfaces, prototyping, simulation, and test data generation.

Operators and maintenance personnel (system administration personnel) are responsible for the actual running and maintenance of the hardware and software environment for the database system.



Advantages of Using the DBMS Approach

1. Controlling Redundancy:

Data **redundancy** (such as tends to occur in the "file processing" approach) leads to **wasted storage space**, duplication of effort (when multiple copies of a data need to be updated), and a higher likelihood of the introduction of inconsistency. On the other hand, redundancy can be used to improve performance of queries. Indexes, for example, are entirely redundant, but help the DBMS in processing queries more quickly.

A DBMS should provide the capability to automatically enforce the rule that no inconsistencies are introduced when data is updated.

2. Restricting Unauthorized Access:

A DBMS should provide a security and authorization subsystem, which the DBA uses to create accounts and to specify account restrictions. Then, the DBMS should enforce these restrictions automatically.

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Smith	112	MATH2410	B
17	Smith	119	CS1310	C
8	Brown	85	MATH2410	A
8	Brown	92	CS1310	A
8	Brown	102	CS3320	B
8	Brown	135	CS3380	A

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Brown	112	MATH2410	B

3. Providing Persistent Storage for Program Objects:

Object-oriented database systems make it easier for complex runtime objects (e.g., lists, trees) to be saved in secondary storage so as to survive beyond program termination and to be retrievable at a later time.

4. Providing Storage Structures and Search Techniques for Efficient Query Processing:

Database systems must provide capabilities for efficiently executing queries and updates. **The query processing and optimization** module of the DBMS is responsible for choosing an efficient query execution plan for each query based on the existing storage structures.



5. Providing Backup and Recovery:

A DBMS must provide facilities for recovering from hardware or software failures. The backup and recovery subsystem of the DBMS is responsible for recovery.

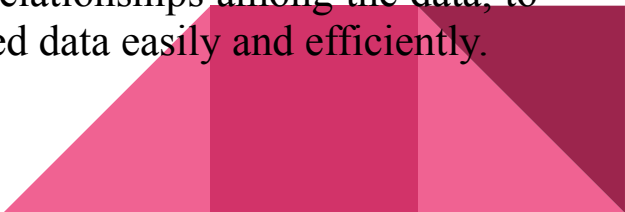
The recovery subsystem could ensure that the transaction is resumed from the point at which it was interrupted so that its full effect is recorded in the database.

6. Providing Multiple User Interfaces:

Many types of users with varying levels of technical knowledge use a database, a DBMS should provide a variety of user interfaces. For example, query languages for casual users, programming language interfaces for application programmers, forms and/or command codes for parametric users, menu-driven interfaces for stand- alone users.

7. Representing Complex Relationships Among Data:

A DBMS must have the capability to represent a variety of complex relationships among the data, to define new relationships as they arise, and to retrieve and update related data easily and efficiently.



8. Enforcing Integrity Constraints:

Most database applications have certain integrity constraints that must hold for the data. The simplest type of integrity constraint involves specifying a **data type** for each data item.

A more complex type of constraint that frequently occurs involves specifying that a record in one file must be related to records in other files. This is known as a **referential integrity constraint**.

Another type of constraint specifies uniqueness on data item values, this is known as a **key** or **uniqueness constraint**.

9. Permitting Inferencing and Actions Via Rules:

In a deductive database system, one may specify declarative rules that allow the database to infer new data!

E.g., Figure out which students are on academic probation. Such capabilities would take the place of application programs that would be used to ascertain such information otherwise.

Active database systems go one step further by allowing "active rules" that can be used to initiate actions automatically.



A Brief History of Database Applications

- Early Database Applications:
 - The Hierarchical and Network Models were introduced in mid 1960s and dominated during the 70's
 - A bulk of the worldwide database processing still occurs using these models
- Relational Model based Systems:
 - Relational model was originally introduced in 1970, was heavily researched and experimented with in
 - IBM Research and several universities

Object-oriented and emerging applications: Object-Oriented Database Management Systems (OODBMSs) were introduced in late 1980s and early 1990s to cater to the need of complex data processing in CAD and other applications.

- Their use has not taken off much.

Many relational DBMSs have incorporated object database concepts, leading to a new category called object-relational DBMSs (ORDBMSs)

Extended relational systems add further capabilities (e.g. for multimedia data, XML, and other data types)

- Relational DBMS Products emerged in the 1980s
- Data on the Web and E-commerce Applications
- Web contains data in HTML (Hypertext markup language) with links among pages.
- This has given rise to a new set of applications and E-commerce is using new standards like XML (eXtended Markup Language)



- Script programming languages such as PHP and JavaScript allow generation of dynamic Web pages that are partially generated from a database
- New functionality is being added to DBMSs in the following areas:
 - Scientific Applications
 - XML (extensible Markup Language)
 - Image Storage and Management
 - Audio and Video data management
 - Data Warehousing and Data Mining
 - Spatial data management
 - Time Series and Historical Data Management
 - The above gives rise to new research and development in incorporating new data types, complex data structures, new operations and storage and indexing schemes in database systems.

When Not to Use a DBMS

- Main inhibitors (costs) of using a DBMS:

High initial investment and possible need for additional hardware.

Overhead for providing generality, security, concurrency control, recovery, and integrity functions.

- When a DBMS may be unnecessary:

If the database and applications are simple, well defined, and not expected to change.

If there are stringent real-time requirements that may not be met because of DBMS overhead.

If access to data by multiple users is not required.

- When no DBMS may suffice:

If the database system is not able to handle the complexity of data because of modeling limitations.

If the database users need special operations not supported by the DBMS.



QUESTIONS

1. Define the following terms:
 - i. Data and database
 - ii. DBMS
 - iii. Database catalog and meta-data
 - iv. Canned transaction
 - v. Persistent stored object
 2. Discuss the main characteristics of the database approach and how it differs from traditional file systems.
 3. What are the responsibilities of the DBA and the database designers?
 4. What are the different types of database end users? Discuss the main activities of each.
 5. Discuss about main advantages of Database system over traditional file processing system.
 6. List and describe roles of workers behind the scene.
 7. Describe the advantages of DBMS.
- 

OVERVIEW OF DATABASE LANGUAGES AND ARCHITECTURES



Data Models, Schemas, and Instances

Data abstraction generally refers to the **suppression of details of data organization** and storage, and the highlighting of the essential features for an improved understanding of data.

One of the main characteristics of the database approach is to support data abstraction so that different users can perceive data at their preferred level of detail.

A **data model**—a collection of concepts that can be used to describe the structure of a database—provides the necessary means to achieve this abstraction



Categories of Data Models

1. High-level or conceptual data models

- provide concepts that are close to the way many users perceive data
- use concepts such as entities, attributes, and relationships.

An **entity** represents a real-world object or concept, An **attribute** represents some property of interest that further describes an entity, A **relationship** among two or more entities represents an association among the entities.

Entity-relationship model (ER) is popular high-level conceptual data model.

2. Low-level or physical data models

- provide concepts that describe the details of **how data is stored on the computer storage media**, typically magnetic disks.
- Concepts provided by low-level data models are generally meant for **computer specialists**, not for end users



3. Representational (or implementation) data models

- provide concepts that may be **easily understood by end users** but that are not too far removed from the way data is organized in computer storage
- in between high level and low level
- hide many details of data storage on disk but can be implemented on a computer system directly
- **Record-based model**



Relation name

Attributes

STUDENT

ROLL_NO

NAME

AGE

PHONE

1

David

18

2525364236

2

Benjamin

19

3526699663

Tuples



Schemas, Instances, and Database State

One must distinguish between the **description** of a database and the database itself. The former is called the **database schema**, which is specified during design and is not expected to change often.

Application requirements change occasionally, which is one of the reasons why software maintenance is important. On such occasions, a change to a database's schema may be called for. Making changes to a database schema is known as **schema evolution**.

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

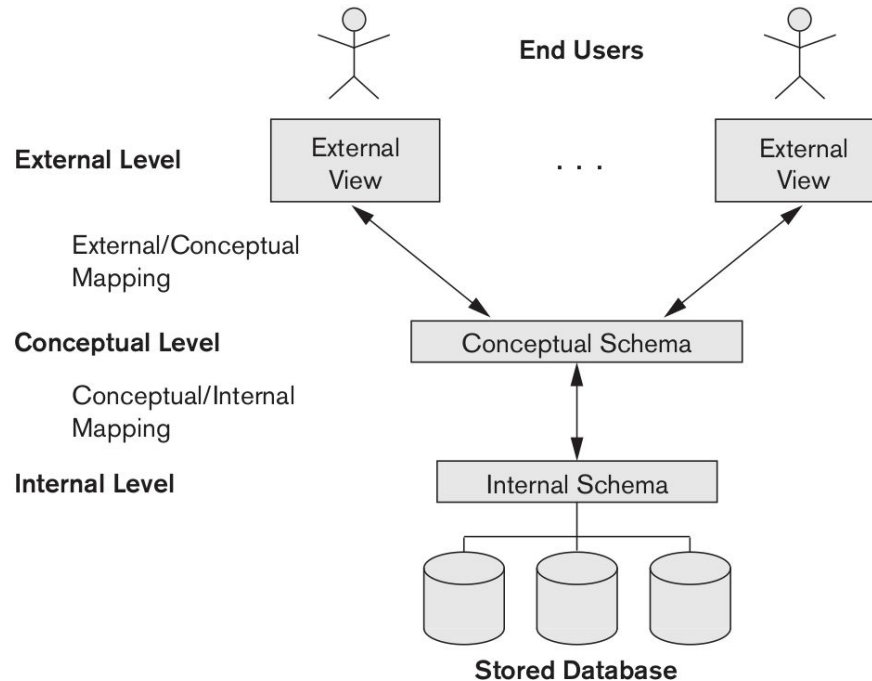
Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

DBMS Architecture and Data Independence

Three-Schema Architecture: The goal is to separate user applications and the physical database.



Internal level: has an internal/physical schema that describes the physical storage structure of the database using a low-level data model)

Conceptual level: has a conceptual schema describing the (logical) structure of the **whole database** for a community of users. It hides physical storage details, concentrating upon describing entities, data types, relationships, user operations, and constraints. Can be described using either high-level or implementational data model.

External/view level: includes a number of external schemas (or user views), each of which describes part of the database that a particular category of users is interested in, hiding rest of database. Can be described using either high-level or implementational data model. (In practice, usually described using same model as is the conceptual schema)

Users (including application programs) submit queries that are expressed with respect to the external level. It is the responsibility of the DBMS to **transform** such a query into one that is expressed with respect to the internal level (and to transform the result, which is at the internal level, into its equivalent at the external level).

By virtue of mappings between the levels:

- external/conceptual mapping (providing logical data independence)
- conceptual/internal mapping (providing physical data independence)



Data Independence

Data independence, which can be defined as the **capacity to change the schema** at one level of a database system without having to change the schema at the next higher level. We can define two types of data independence:

1. Logical data independence

- capacity to change the conceptual schema without having to change external schemas or application programs
- change the conceptual schema to expand the database, to change constraints, or to reduce the database changes to constraints can be applied to the conceptual schema without affecting the external schemas or application programs

2. Physical data independence

- capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas need not be changed as well by creating additional access structures—to improve the performance of retrieval or update.

Database Languages and Interfaces

DBMS Languages

DBMS packages provide an integrated feature of above languages into a single language called **Structured Query Language**.

Data definition language (DDL), is used by the DBA and by database designers to define both schemas.

Storage definition language (SDL), is used to specify the internal schema.

View definition language (VDL), to specify user views and their mappings to the conceptual schema.

Data manipulation language(DML) provides set of operations like retrieval, insertion, deletion, and modification of the data.



A high-level or non procedural DML

- can be used on its own to specify **complex database** operations concisely many DBMS
- high-level DML statements either to be entered interactively from a display monitor or terminal or to be embedded in a general-purpose programming language.
- can specify and retrieve many records in a single DML statement; therefore, they are called **set-at-a-time** or **set-oriented DMLs**
- declarative language

A low-level or procedural DML

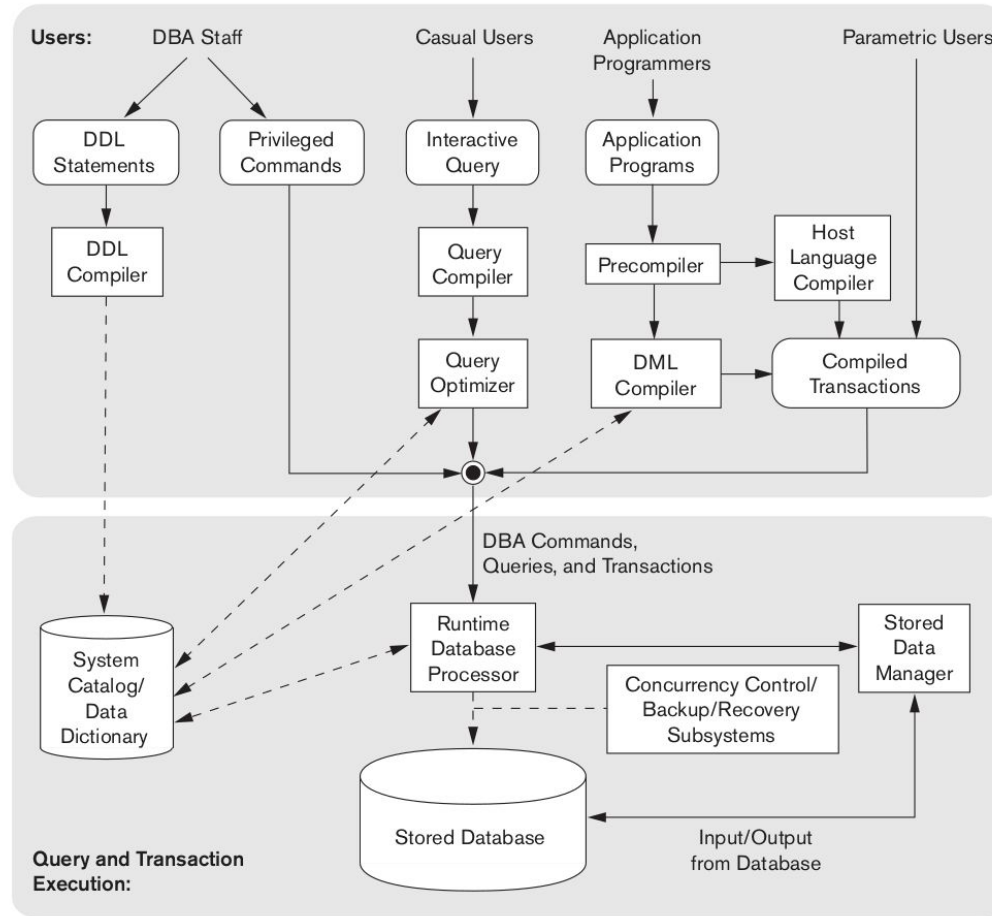
- must be embedded in a general-purpose programming language
- retrieves individual records or objects from the database and processes each separately
- to retrieve and process each record from a set of records. Low-level DMLs are also called **record-at-a-time DMLs**



DBMS Interfaces

- **Menu-Based Interfaces for Web Clients or Browsing.** These interfaces present the user with lists of options (called menus) that lead the user through the formulation of a request.
 - **Forms-Based Interfaces** displays a form to each user. Users can fill out all of the form entries to insert new data, or they can fill out only certain entries, in which case the DBMS will retrieve matching data for the remaining entries.
 - **Graphical User Interfaces** displays a schema to the user in diagrammatic form. The user then can specify a query by manipulating the diagram. GUIs utilize both menus and forms.
 - **Natural Language Interfaces** accepts requests written in English or some other language and attempt to understand them.
 - **Speech Input and Output** use of speech as an input query and speech as an answer to a question or result. The speech input is detected using a library of predefined words and used to set up the parameters that are supplied to the queries.
 - **Interfaces for Parametric Users** such as bank tellers, often have a small set of operations that they must perform repeatedly.
 - **Interfaces for the DBA.** DBA use privileged commands. These include commands for creating accounts, setting system parameters, granting account authorization, changing a schema, and reorganizing the storage structures of a database.
- 

The Database System Environment



- The figure is divided into two parts.
- The **top part** of the figure refers to the various users of the database environment and their interfaces. The **lower part** shows the internals of the DBMS responsible for storage of data and processing of transactions.
- The database and the DBMS catalog are usually stored on disk. Access to the disk is controlled primarily by the operating system (OS), which schedules disk read/write.
- Many DBMSs have their own **buffer management** module to schedule disk read/write.
- **Stored data manager** controls access to DBMS information that is stored on disk, whether it is part of the database or the catalog.

Top part:

- it shows interfaces for the DBA staff, casual users who work with interactive interfaces to formulate queries
 - application programmers who create programs using some host programming languages
 - parametric users who do data entry work by supplying parameters to predefined transactions.
 - the DBA staff works on defining the database and tuning it by making changes to its definition using the DDL and other privileged commands
- 

In the **lower part** of Figure,

- the runtime database processor executes
 - the privileged commands
 - the executable query plans, and
 - the canned transactions with runtime parameters.
 - It works with the system catalog and may update it with statistics
 - It also works with the stored data manager, which in turn uses basic operating system services for carrying out
 - low-level input/output (read/write) operations between the disk and main memory
 - The runtime database processor handles other aspects of data transfer, such as management of buffers concurrency control and backup and recovery systems, integrated into the working of the runtime database processor for purposes of transaction management
- 

Database System Utilities

DBMSs have database utilities (Program) that help the DBA manage the database system.

Common utilities have the following types of functions:

Loading

- used to load existing data files—such as text files or sequential files—into the database
- automatically reformats the data and stores it in the database
- for loading programs, conversion tools are available like IDMS (Computer Associates), SUPRA (Cincom), and IMAGE (HP)

Backup

- creates a backup copy of the database, by dumping the entire database onto tape
 - used to restore the database in case of catastrophic disk failure
 - Incremental backups are also often used, to save space
- 

Database storage reorganization

- used to reorganize a set of database files into different file organizations to improve performance

Performance monitoring

- monitors database usage and provides statistics to the DBA.
- Statistics used for making decisions



Tools, Application Environments, and Communications Facilities

CASE (Computer-Aided Software Engineering Tools) **tools** are used in the design phase of database systems

Data dictionary (or data repository) system for storing catalog information about schemas and constraints

Information repository stores information such as design decisions, usage standards, application program descriptions, and user information

Application development environment systems provide an environment for developing database applications, including database design, GUI development, querying and updating, and application program development

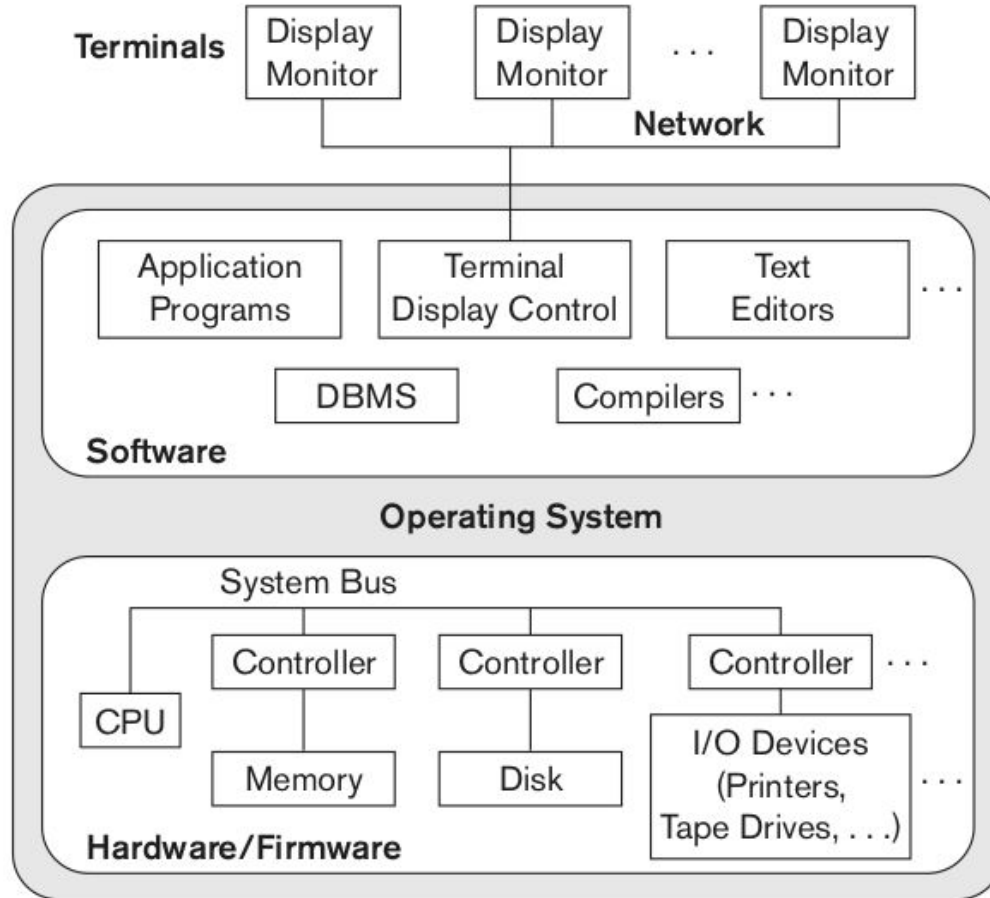
Communications software, whose function is to allow users at locations remote from the database system site to access the database through computer terminals, workstations, or personal computers

These are connected to the database site through data communications hardware such as Internet routers, phone lines, long-haul networks, local networks, or satellite communication devices

The integrated DBMS and data communications system is called a DB/DC system



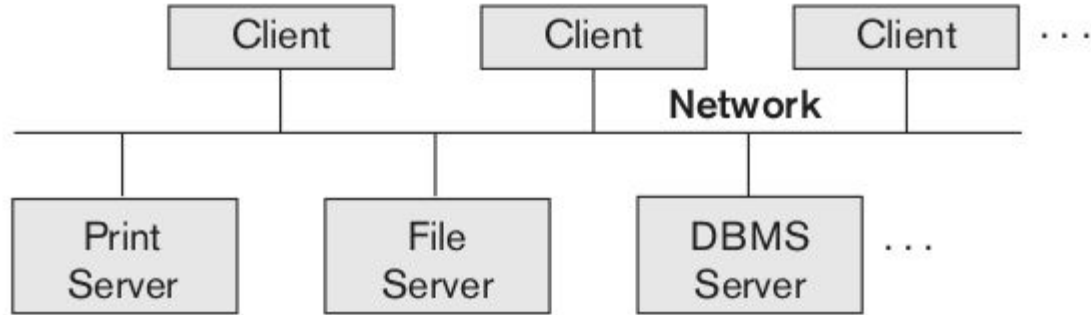
Centralized DBMSs Architecture



- Earlier architectures used mainframe computers to provide the main processing for all system functions
- These mainframes replaced by users with their terminals with PCs and workstations
- DB systems used these computers similarly to how they had used display terminals
- So that the DBMS itself was still a **centralized DBMS** in which all the DBMS functionality, application program execution, and user interface processing were carried out on one machine
- Gradually, DBMS systems started to exploit the available processing power at the user side, which led to **client/server DBMS architectures**



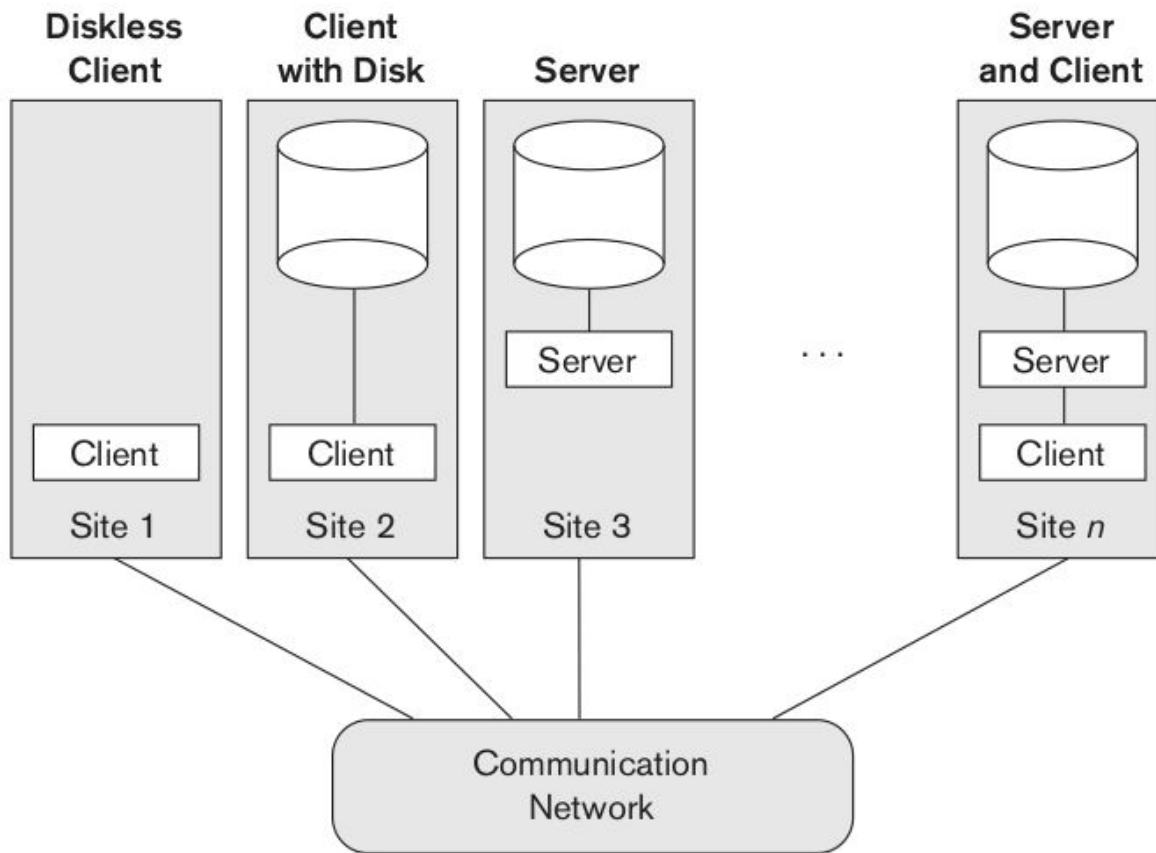
Basic Client/Server Architectures



- The client/server architecture was developed to deal with computing environments in which a large number of PCs, workstations, file servers, printers, database servers, Web servers, e-mail servers, and other software and equipment are connected via a network.
- The idea is to **define specialized servers** with **specific functionalities**
- It is possible to connect a number of PCs or small workstations as clients to a **file server** that maintains the files of the client machines
- Another machine can be designated as a **printer server** by being connected to various printers; all print requests by the clients are forwarded to this machine
- **Web servers** or **e-mail servers** also fall into the specialized server category. The resources provided by specialized servers can be accessed by many client machines
- The client machines provide the user with the appropriate interfaces to utilize these servers, as well as with local processing power to run local applications.
- This concept can be carried over to other software packages, with specialized programs—such as a CAD (computer-aided design) package



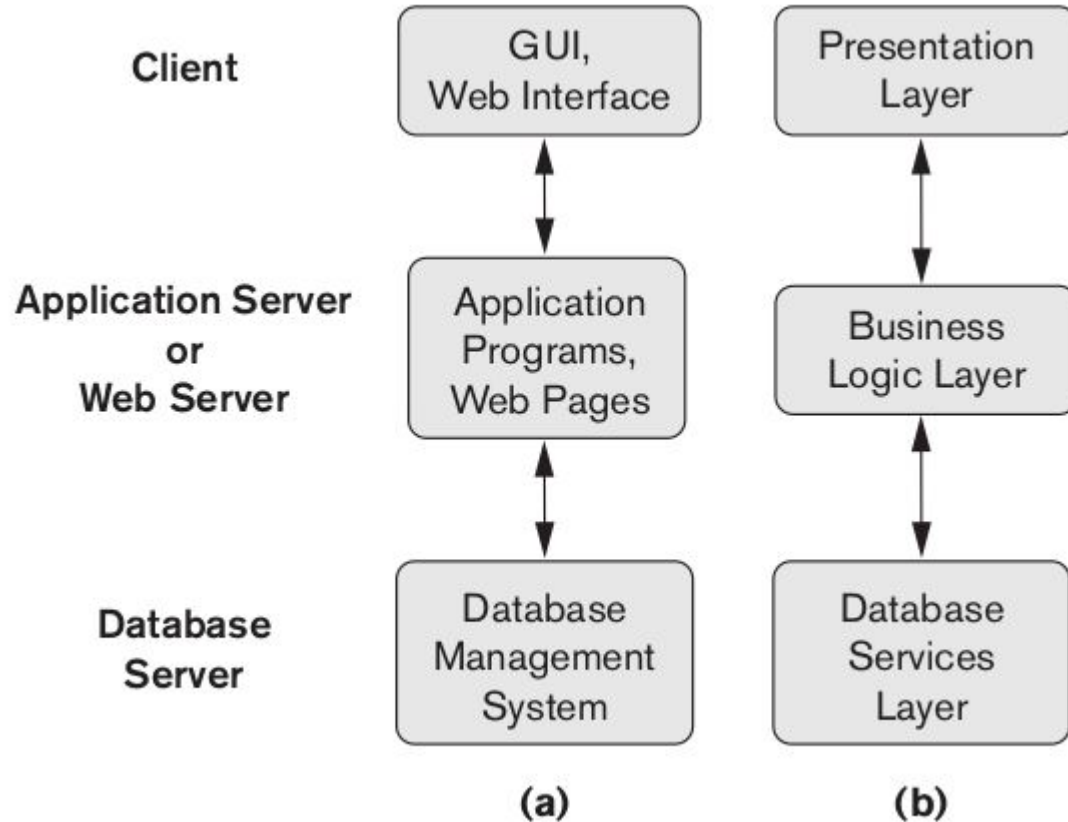
Physical two-tier client/server architecture



- Some machines would be **client sites only**, other machines would be dedicated servers, and others would have both client and server functionality
- The concept of **client/server architecture** assumes an underlying framework that consists of many PCs and workstations as well as a smaller number of mainframe machines, connected via LANs and other types of computer networks
- A client machine provides user machine that provides user interface capabilities and local processing
- A **server** is a system containing both hardware and software that can provide services to the client machines, such as file access, printing, archiving, or database access.
- In general, some machines install only client software, others only server software, and still others may include both client and server software



Three-Tier and n-Tier Architectures for Web Applications



- Many Web applications use an architecture called the three-tier architecture, which adds an intermediate layer between the client and the database server
 - This **intermediate layer or middle tier** is called the application server or the Web server, depending on the application
 - This server plays an intermediary role by running application programs and storing business rules (procedures or constraints) that are used to access data from the database server
 - It can also improve database security by checking a client's credentials before forwarding a request to the database server
 - Clients contain GUI interfaces and some additional application-specific business rules
 - The intermediate server accepts requests from the client, processes the request and sends database queries and commands to the database server, and then acts as a conduit for passing (partially) processed data from the database server to the clients
 - Thus, the user interface, application rules, and data access act as the three tiers
- 

- The **presentation layer** displays information to the user and allows data entry
- The **business logic layer** handles intermediate rules and constraints before data is passed up to the user or down to the DBMS
- The **bottom layer** includes all data management services. The middle layer can also act as a Web server, which retrieves query results from the database server and formats them into dynamic Web pages that are viewed by the Web browser at the client side
- If business logic layer is divided into multiple layer, then called as n-tier architecture



Classification of Database Management Systems

Data Model

- Used in commercial DBMS [eg: relational data model, object data model]
- Many legacy applications still run on database systems based on the **hierarchical** and **network data models**

Number of users

- **Single-user systems** support only one user at a time and are mostly used with PCs
- **Multiuser systems**, which include the majority of DBMSs, support concurrent multiple users

Number of sites

- **Centralized DBMS** : the data is stored at a single computer site
- **Distributed DBMS [DDBMS]** : DBMS software distributed over many sites



Cost

- Open source like MYSQL & PostgreSQL
- 30 day copy versions
- Sold in form of licenses

Types of access path

- inverted file structures
- general purpose or special purpose
- online transaction processing (OLTP) system

Other important data models

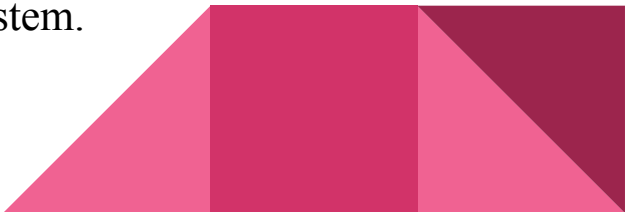
Network model

- represents data as record types and also represents a limited type of 1:N relationship, called a set type

Hierarchical model

- represents data as hierarchical tree structures
 - Each hierarchy represents a number of related records
- 

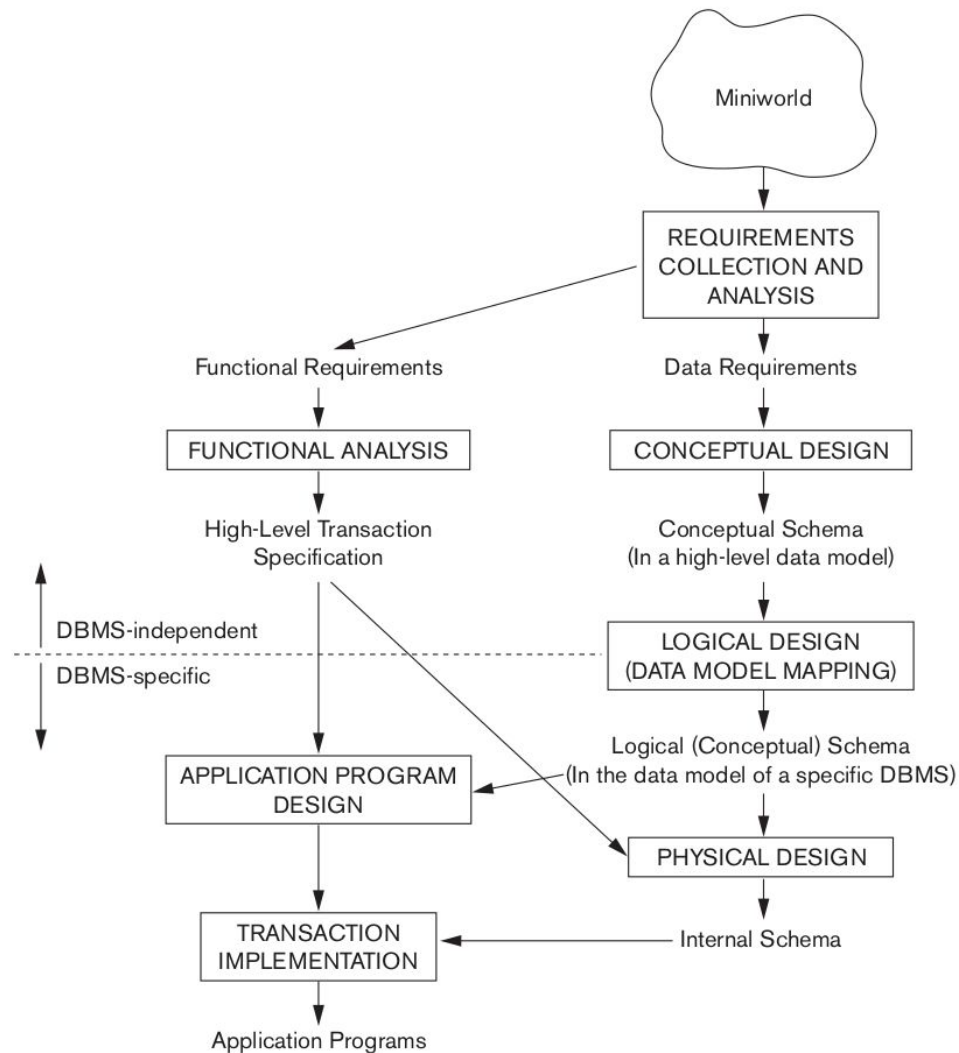
QUESTIONS

1. Define the following:
 - a. Data model
 - b. Schema and snapshot
 - c. Intension and extension
 2. With neat diagram, explain three schema architecture along with its disadvantage.
 3. Differentiate between physical data independence and logical data independence.
 4. List and explain different types of DBMS interfaces.
 5. With neat diagram, explain the components module of database system.
 6. Differentiate two-tier and three tier client-server architectures.
- 

Conceptual Data Modeling and Database Design



Main phases of Database Design.



Conceptual modeling is a very important phase in **designing a successful** database application.

The main phases of database design are:

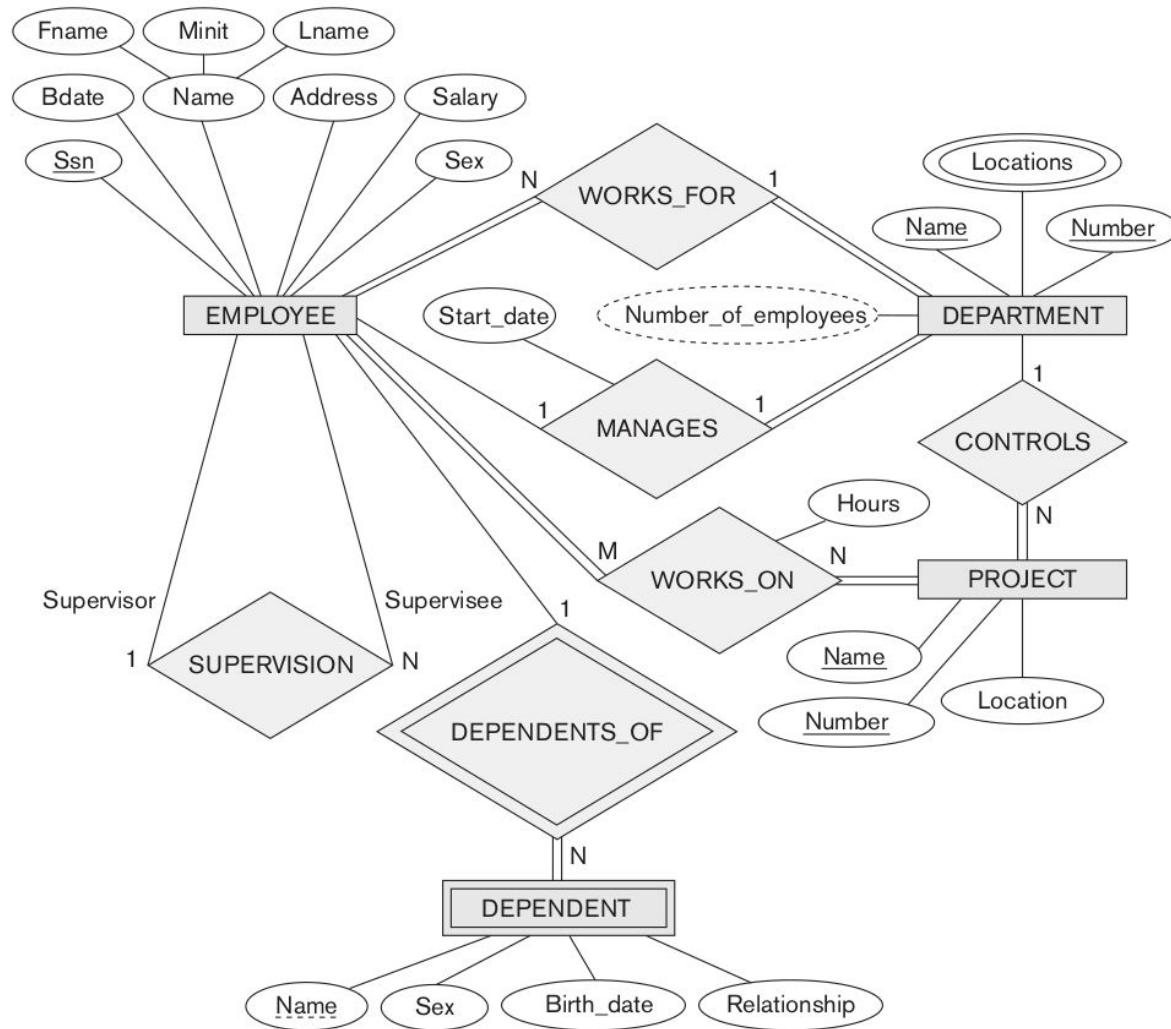
Requirements Collection and Analysis: purpose is to produce a description of the users' requirements.

Conceptual Design: purpose is to produce a conceptual schema for the database, including detailed descriptions of entity types, relationship types, and constraints. All these are expressed in terms provided by the data model being used. Eg: ER model

Logical Design: purpose is to transform the conceptual schema (which is at a high/abstract level) into a (lower-level) representational/implementational model supported by whatever DBMS is to be used.

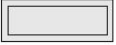
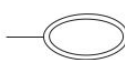
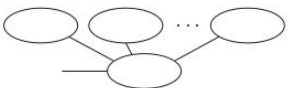
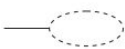
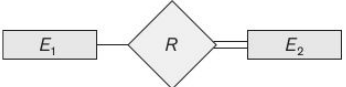
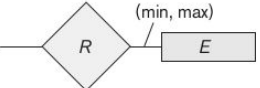
Physical Design: purpose is to decide upon the internal storage structures, access paths (indexes), etc., that will be used in realizing the representational model produced in previous phase.





1. The company is organized into departments. Each department has a unique name, a unique number, and a particular employee who manages the department. We keep track of the start date when that employee began managing the department. A department may have several locations.
2. A department controls a number of projects, each of which has a unique name, a unique number, and a single location.
3. The database will store each employee's name, Social Security number, address, salary, sex (gender), and birth date. An employee is assigned to one department, but may work on several projects, which are not necessarily controlled by the same department. It is required to keep track of the current number of hours per week that an employee works on each project, as well as the direct supervisor of each employee (who is another employee).
4. The database will keep track of the dependents of each employee for insurance purposes, including each dependent's first name, sex, birth date, and relationship to the employee.



Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1:N for $E_1 : E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

Relation name

Attributes

STUDENT	ROLL_NO	NAME	AGE	PHONE
1	David	18	2525364236	
2	Benjamin	19	3526699663	

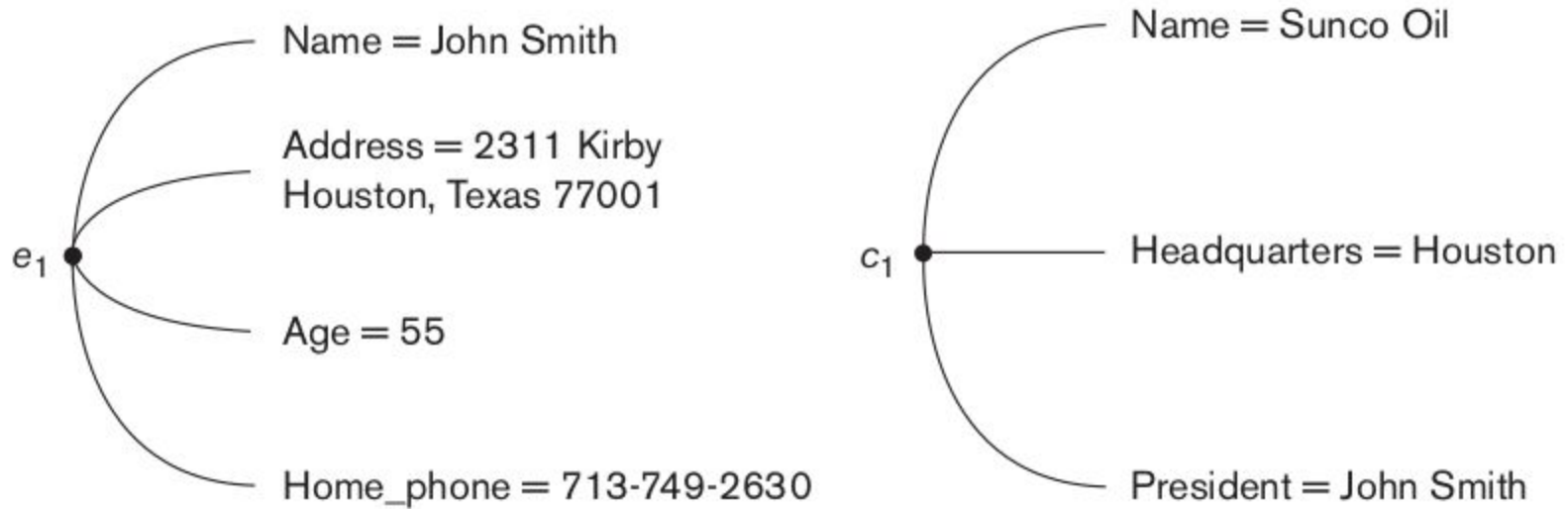
1. The basic concept that the ER model represents is an **entity**, which is a thing or object in the real world with an independent existence. An entity may be an object with a **physical existence** (for example, a particular person, car, house, or employee) or it may be an object with a **conceptual existence** (for instance, a company, a job, or a university course).
2. Each entity has **attributes**—the particular properties that describe it. For example, an EMPLOYEE entity may be described by the employee's name, age, address, salary, and job.

Types of attributes occur in the ER model

- Simple versus Composite
- Single valued versus Multivalued
- Stored versus Derived
- Complex Attributes.

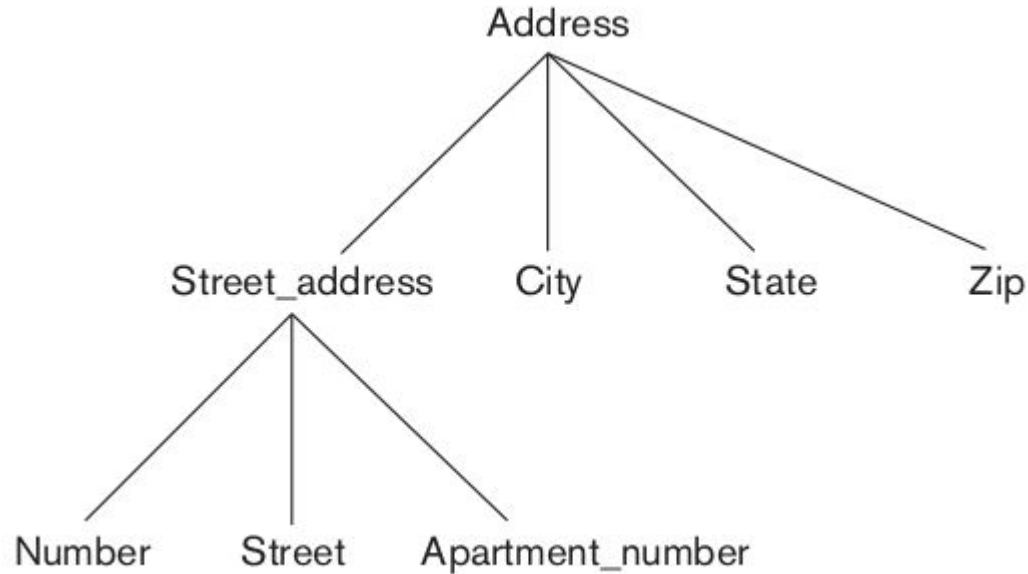


Two entities, **EMPLOYEE** e_1 , and **COMPANY** c_1 , and their attributes.



Composite versus Simple (Atomic) Attributes.

The value of a composite attribute is the concatenation of the values of its component simple attributes.



Composite attributes can be divided into smaller subparts.

For eg: the Address attribute of the EMPLOYEE entity can be subdivided into Street_address, City, State, and Zip, 3 with the values '2311 Kirby', 'Houston', 'Texas', and '77001'.

Attributes that are not divisible are called **simple or atomic attributes**.

For eg: attribute Age cannot be divided



Single-valued - Most attributes have a single value for a particular entity

For eg: Age is a single-valued attribute of a person

Multivalued - An entity having multiple values for that attribute

For eg: color of a color color={black,red} Person's degree degree={BE, MTech, PhD}

Stored and Derived attribute

Two (or more) attribute values are related—for eg: the Age and Birth_date attributes of a person

The value of Age can be determined from the current (today's) date and the value of that person's

Birth_date

The **Age** attribute is hence called a derived attribute

Birth_date attribute is called a stored attribute



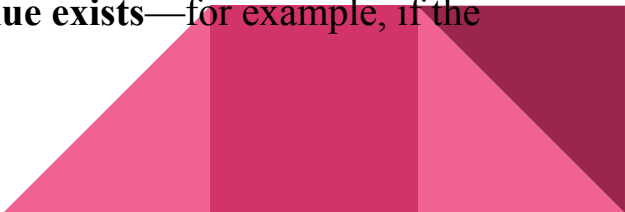
NULL Values

In some cases, a **particular entity may not have an applicable value for an attribute**. eg: the **Apartment_number attribute** of an address applies only to addresses that are in apartment buildings and not to other types of residences, such as single-family homes, College_degrees attribute applies only to people with college degrees.

NULL can also be used if **we do not know the value of an attribute for a particular entity**— for example, if we do not know the home phone number of a person.

The meaning of the former type of NULL is **not applicable**, whereas the meaning of the latter is **unknown**.

The **unknown category of NULL** can be further classified into **two cases**.

1. The first case arises when it is known that the attribute value exists but is **missing**—for instance, if the Height attribute of a person is listed as NULL.
 2. The second case arises **when it is not known whether the attribute value exists**—for example, if the Home_phone attribute of a person is NULL.
- 

Complex Attributes

Composite and multivalued attributes can be nested arbitrarily

Arbitrary nesting by grouping components of a **composite attribute between parentheses ()** and separating the components with commas, and by displaying **multivalued attributes between braces { }**. Such attributes are called **complex attributes**

For example, if a person can have more than one residence and each residence can have a single address and multiple phones, an attribute Address_phone for a person

```
{Address_phone( {Phone(Area_code,Phone_number)},Address(Street_address  
(Number,Street,Apartment_number),City,State,Zip) )}
```



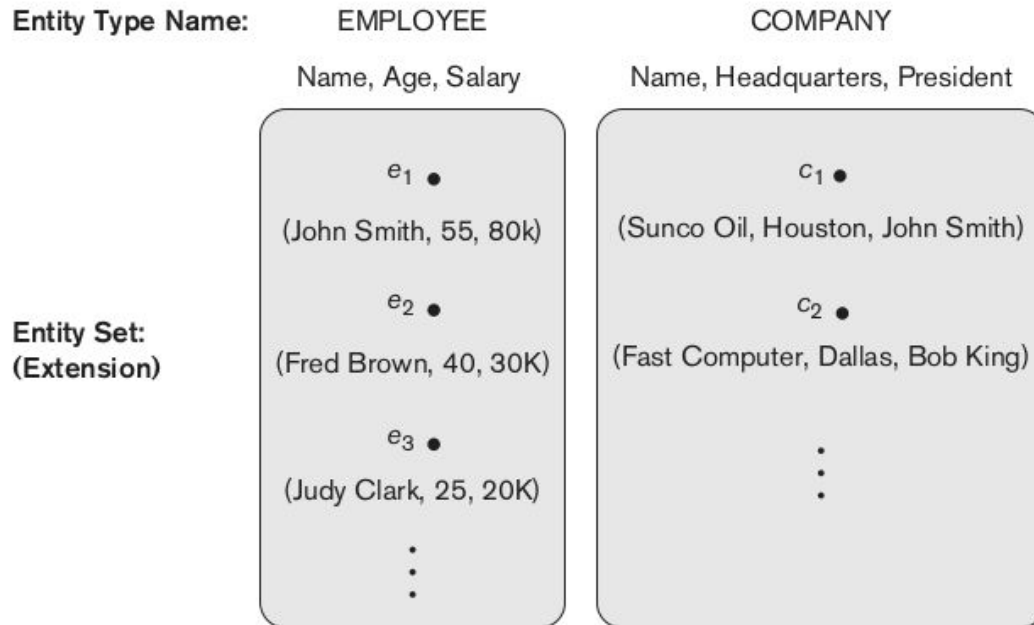
Types of Attribute

	Name	DOB	Address	Age	Phone
Composite Attribute					
Simple/Single-valued					
Complex Attribute					
Derived Attribute					
Multi-valued Attribute					



Entity Types, Entity Sets, Keys, and Value Sets

- **Entity type** defines a collection (or set) of entities that have the same attributes
- Each entity type in the database is described by its name and attributes
- Below figure shows two entity types: EMPLOYEE and COMPANY, and a list of some of the attributes for each



- The collection of all entities of a particular entity type in the database at any point in time is called an **entity set or entity collection**
- Entity set is usually referred to using the same name as the entity type
- An **entity type** is represented in ER diagrams as a **rectangular box**
- **Attribute names** are enclosed in **ovals** and are attached to their entity type by **straight lines**
- **Composite attributes** are attached to their component attributes by **straight lines**
- **Multivalued attributes** are displayed in **double ovals**
- Collection of entities of a particular entity type is grouped into an entity set, which is also called the **extension of the entity type**



Key Attributes of an Entity Type

Important constraint on the entities of an entity type is the **key or uniqueness constraint on attributes**

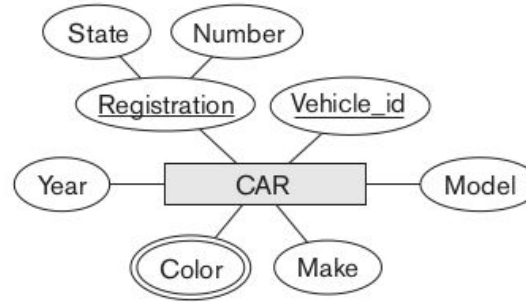
1. An entity type usually has one or more attributes whose values are distinct for each individual entity in the entity set. Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely
2. For eg, the **Name** attribute is a key of the **COMPANY** entity type in because no two companies are allowed to have the same name

For the **PERSON** entity type, a typical key attribute is **Ssn**

1. Specifying that an attribute is a key of an entity type means that the preceding uniqueness property must hold for every entity set of the entity type
 2. Hence, it is a constraint that prohibits any two entities from having the same value for the key attribute at the same time
 3. **Some entity types have more than one key attribute**
- 

For eg, each of the Vehicle_id and Registration attributes of the entity type CAR is a key in its own right

(a)



(b)

CAR
Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR₁
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR₂
((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR₃
((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

⋮

Value Sets (Domains) of Attributes

- Each simple attribute of an entity type is associated with a value set (or domain of values), which specifies the set of values that may be assigned to that attribute for each individual entity
- If the range of ages allowed for employees is between 16 and 70, we can specify the value set of the Age attribute of EMPLOYEE to be the set of integer numbers between 16 and 70
- Value sets are **not typically displayed in basic ER diagrams** and are similar to the basic data types available in most programming languages, such as integer, string, Boolean, float, enumerated type, subrange, and so on

Examples of Value Sets (Domains) for Attributes

Attribute	Value Set (Domain) Example
Age	Integers between 0 and 120
Gender	{Male, Female, Other}
Salary	Decimal values greater than 0
Email	Valid email format (e.g., xyz@example.com)
JoinDate	Date format (YYYY-MM-DD)
Grade	{A, B, C, D, F} (fixed set of letter grades)

Weak Entity Types

- Entity types that **do not have key attributes** of their own are called **weak entity types**
- A **Weak Entity** is an entity type that **cannot exist independently** and relies on a **strong entity** (or owner) for identification.
- In contrast, **regular entity types** that do have a key attribute—are called **strong entity types**
- Entities that belong to a weak entity type are identified by being related to specific entities from another entity type in combination with one of their attribute values. This entity type is called an **identifying or owner entity type**.
- The relationship that relates the identifying entity type with the weak entity type is called an **identifying relationship**.
- A weak entity type always has a **total participation constraint** with respect to the identifying relationship, because a **weak entity cannot exist without its owner**.
- A weak entity type normally has a **partial key**, which is the attribute that can uniquely identify weak entities that are related to the same owner entity. For example, the attribute **Name of DEPENDENT** is the **partial key**.

Dependent (Weak Entity) → Employee (Strong Entity)

- **Scenario:** Employees can have dependents (spouse, children) covered under company benefits.
 - **Weak Entity:** **Dependent** (Cannot exist without an **Employee**).
 - **Strong Entity:** **Employee** (Each dependent belongs to an employee).
 - **Partial Key:** **DependentName** (A name is unique only within an employee but not across all employees).
- 

Project Task (Weak Entity) → Project (Strong Entity)

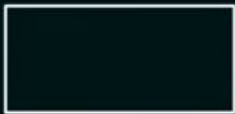
- **Scenario:** Each project consists of multiple tasks assigned to employees.
- **Weak Entity:** **ProjectTask** (A task cannot exist without a **project**).
- **Strong Entity:** **Project** (A project has multiple tasks).
- **Partial Key:** **TaskID** (Unique within a project but not across all projects).



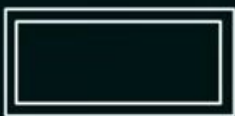
Entity-Relationship (E-R) Diagram

Entity Set

Strong
Entity Set



Weak
Entity Set



Relationship

Strong
Relationship



Weak
Relationship



Attributes

Simple/Single-
Valued



Composite



Multi-Valued



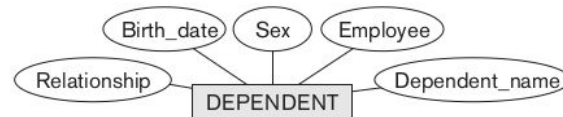
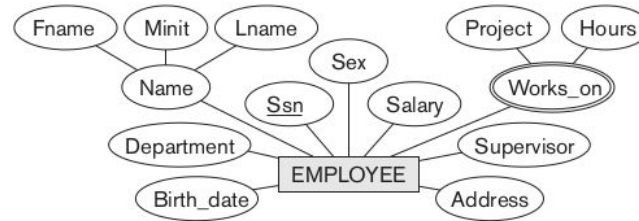
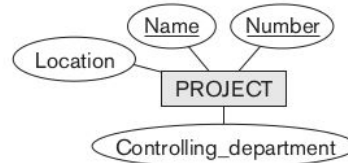
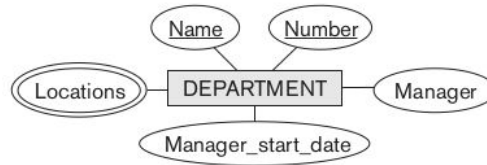
Derived



Null



Initial Conceptual Design of the COMPANY Database



Relationship Types, Relationship Sets, Roles, and Structural Constraints

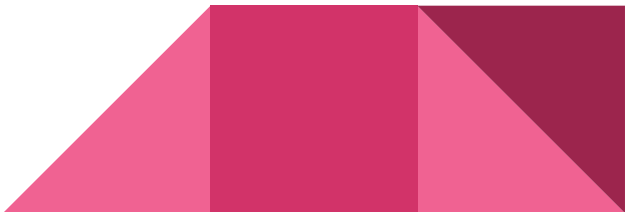
Whenever an attribute of one entity type refers to another entity type, some relationship exists.

For example, the attribute Manager of DEPARTMENT refers to an employee who manages the department, the attribute Controlling_department of PROJECT refers to the department that controls the project in the ER model, **these references should not be represented as attributes but as relationships.**

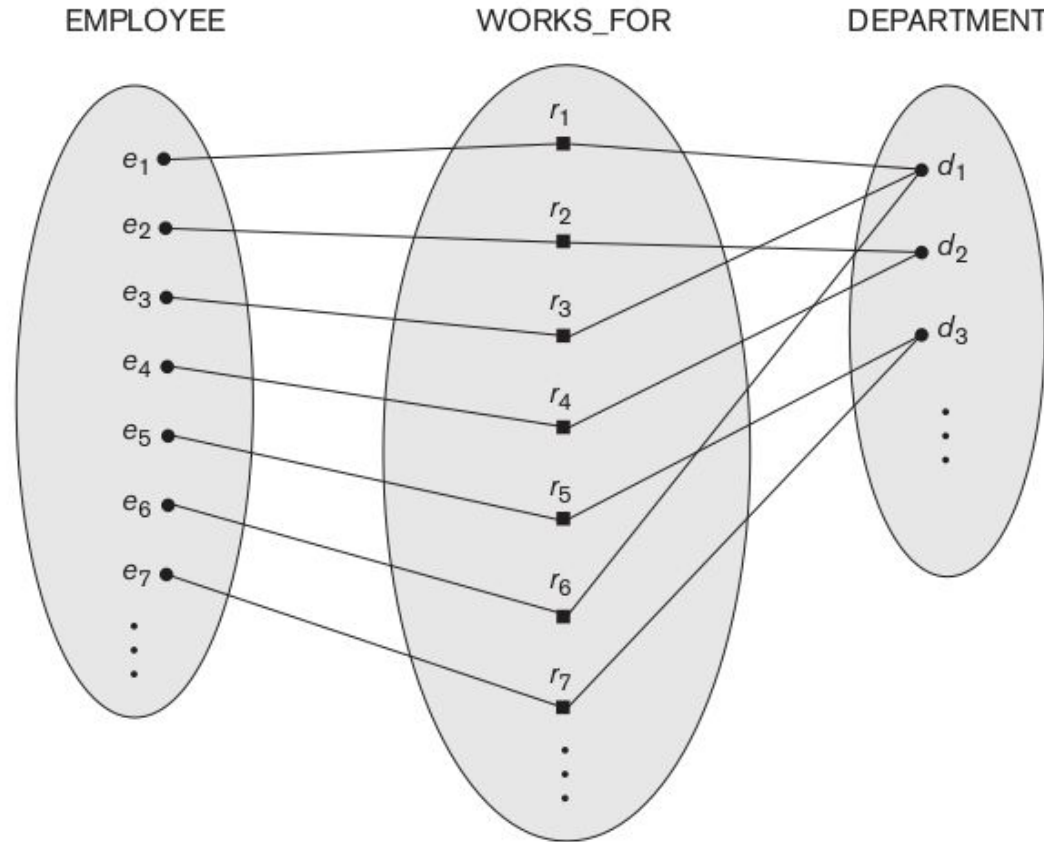


Relationship Types, Sets, and Instances

- A relationship type R among n entity types $E_1, E_2, \dots, E_n$ defines a set of associations—or a relationship set—among entities from these entity types
- Entity types and entity sets, a relationship type and its corresponding relationship set are customarily referred to by the same name, R
- Mathematically, the relationship set R is a set of relationship instances r_i , where each r_i associates n individual entities $(e_1, e_2, \dots, e_n)$, and each entity e_j in r_i is a member of entity set E_j , $1 \leq j \leq n$
- A relationship set is a mathematical relation on $E_1, E_2, \dots, E_n$; alternatively, it can be defined as a subset of the Cartesian product of the entity sets $E_1 \times E_2 \times \dots \times E_n$
- Each of the entity types $E_1, E_2, \dots, E_n$ is said to participate in the relationship type R
- Each of the individual entities $e_1, e_2, \dots, e_n$ is said to participate in the **relationship instance**

$$r_i = (e_1, e_2, \dots, e_n)$$


Some instances in WORKS_FOR relationship set, which represents a relation type WORKS_FOR between EMPLOYEE and DEPARTMENT.



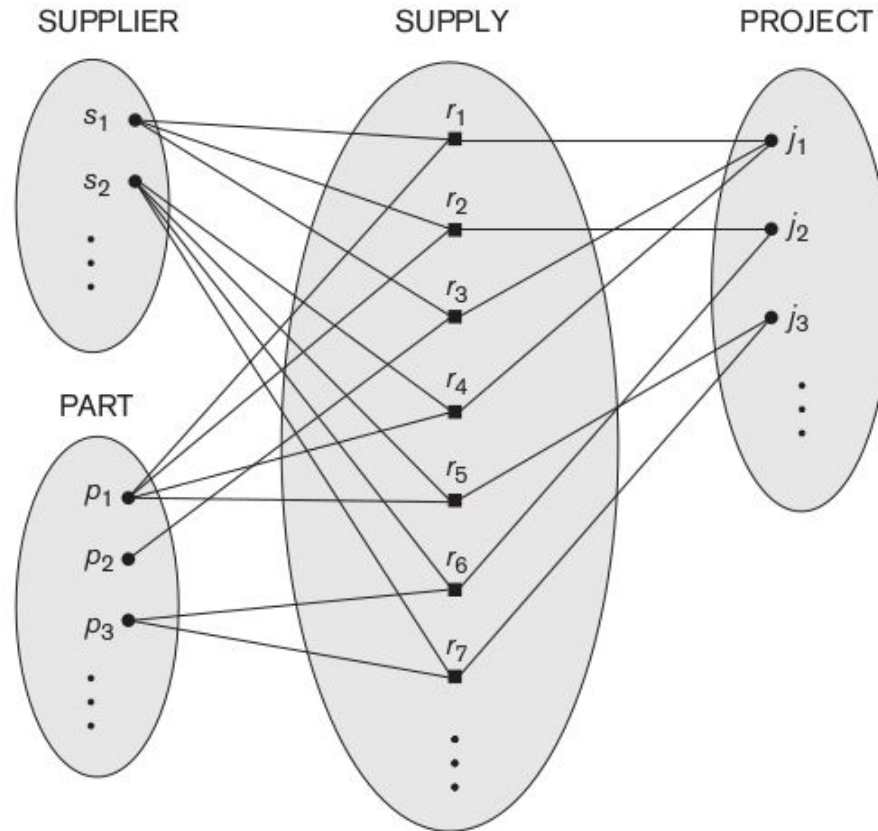
Relationship Degree, Role Names, and Recursive Relationships

Degree of a Relationship Type. The degree of a relationship type is the number of participating entity types.

A relationship type of degree two is called **binary**, and one of degree three is called **ternary**.



Some relationship instances in the SUPPLY ternary relationship set.



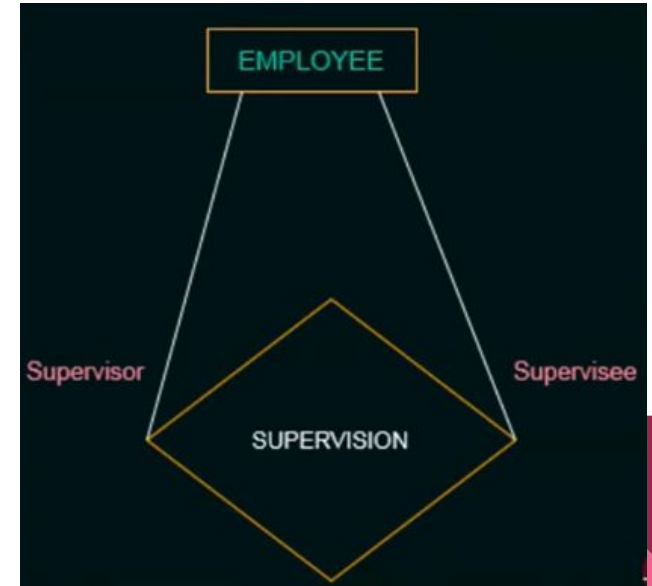
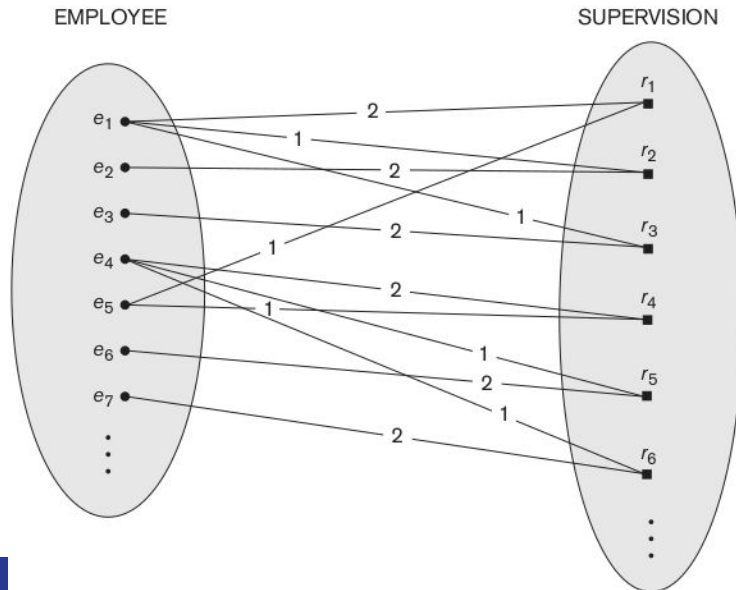
Role Names and Recursive Relationships

- Each entity type in a relationship plays a particular role. The role name specifies the role that a participating entity type plays in the relationship and explains what the relationship means.
- *For example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of employee or worker and DEPARTMENT plays the role of department or employer*
- Same entity type participates more than once in a relationship type in different roles, such relationship types are called **recursive relationships or self-referencing relationships**.



For example, in the Company schema, each employee has a supervisor, we need to include the relationship “Supervises”, however a supervisor is also an employee, therefore the employee entity type participates twice in the relationship, once as an employee and once as a supervisor, and therefore we can specify two roles, employee and supervisor.

A recursive relationship SUPERVISION between EMPLOYEE in the **supervisor role (1)** and EMPLOYEE in the **subordinate role (2)**.



Constraints on Binary Relationship Types

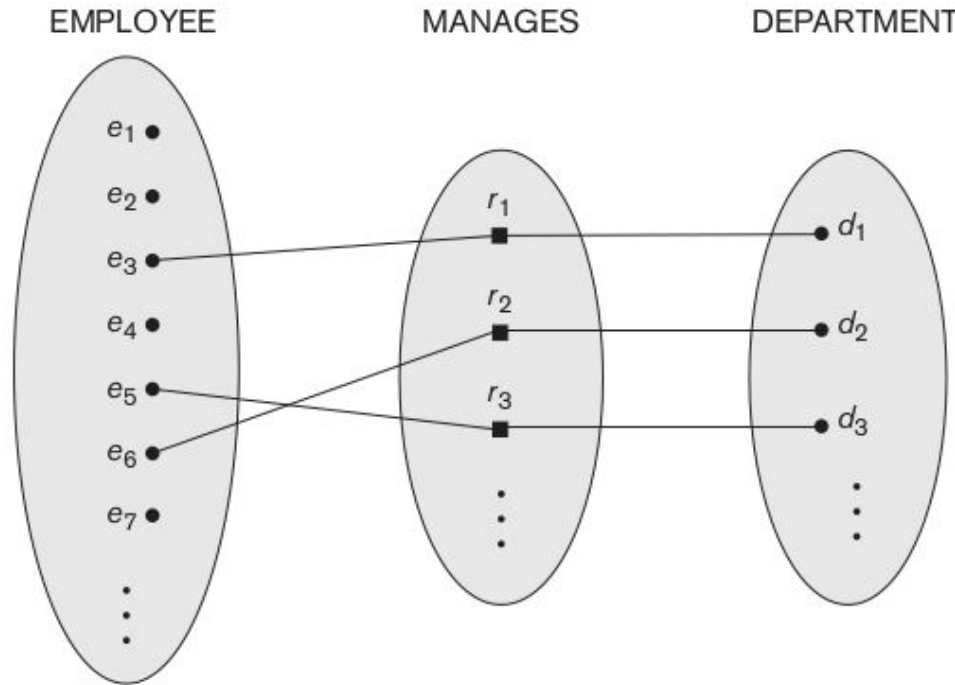
- Relationship types have certain constraints that limit the possible combination of entities that may participate in relationship.
- There are two main types of relationship constraints **cardinality ratio** and **participation**.

Cardinality for Binary Relationship

- The cardinality ratio specifies the maximum number of relationship instances that an entity can participate in.
 - The possible cardinality ratios for binary relationship types are: **1:1, 1:N, N:1, M:N**.
- 

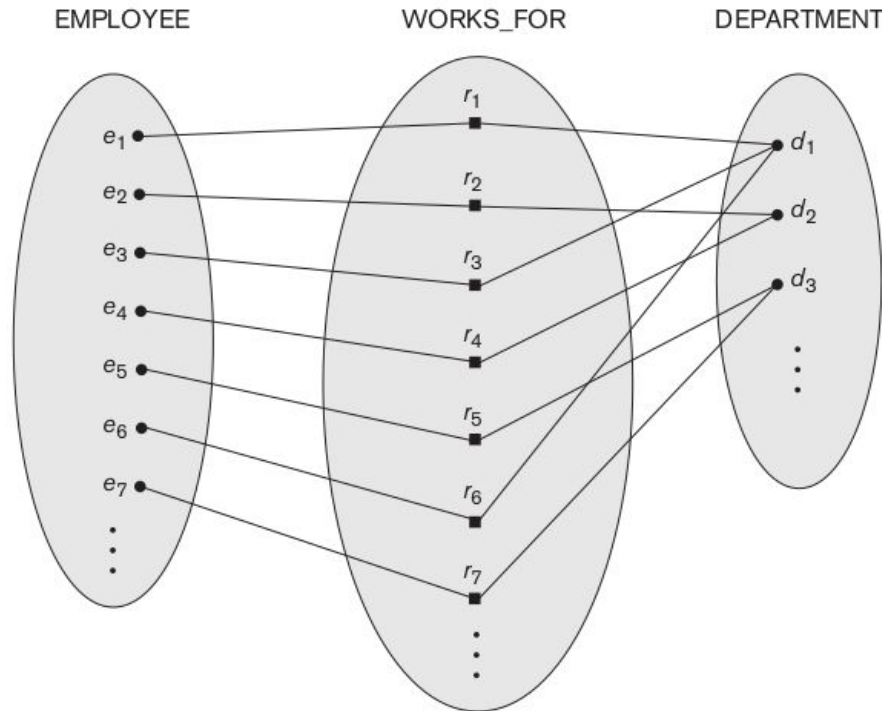
One-to-one – when only one instance of an entity is associated with the relationship, it is marked as '1:1'.

In 1:1 an employee can manage at most one department and a department can have at most one manager



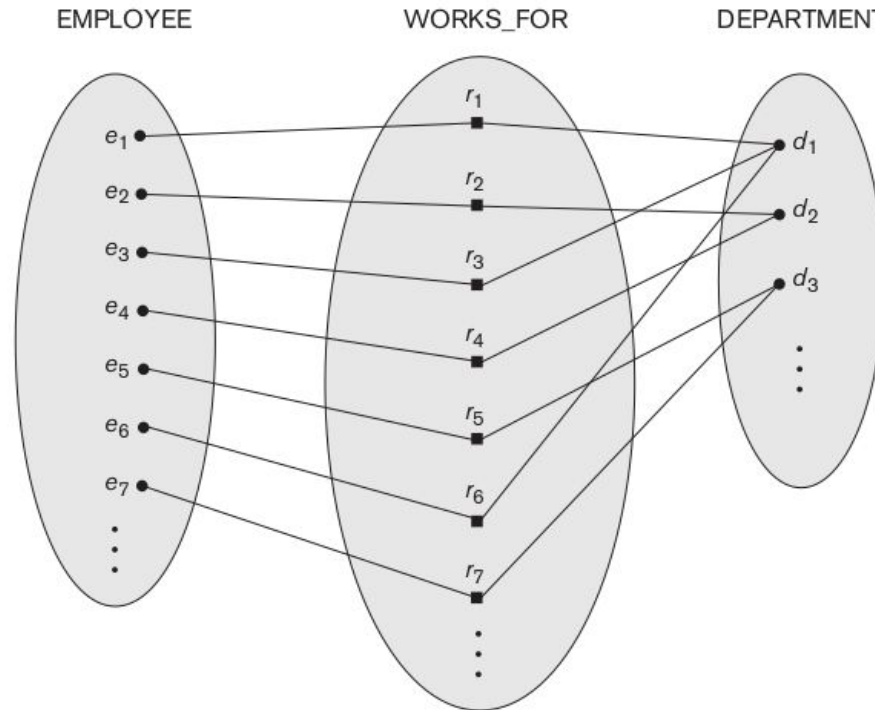
One-to-many – When more than one instance of an entity is associated with a relationship, it is marked as '1:N'.

For example, in the WORKS_FOR binary relationship type, DEPARTMENT: EMPLOYEE is of cardinality ratio 1:N, meaning that each department can have any number of employees, but an employee work for only one department.

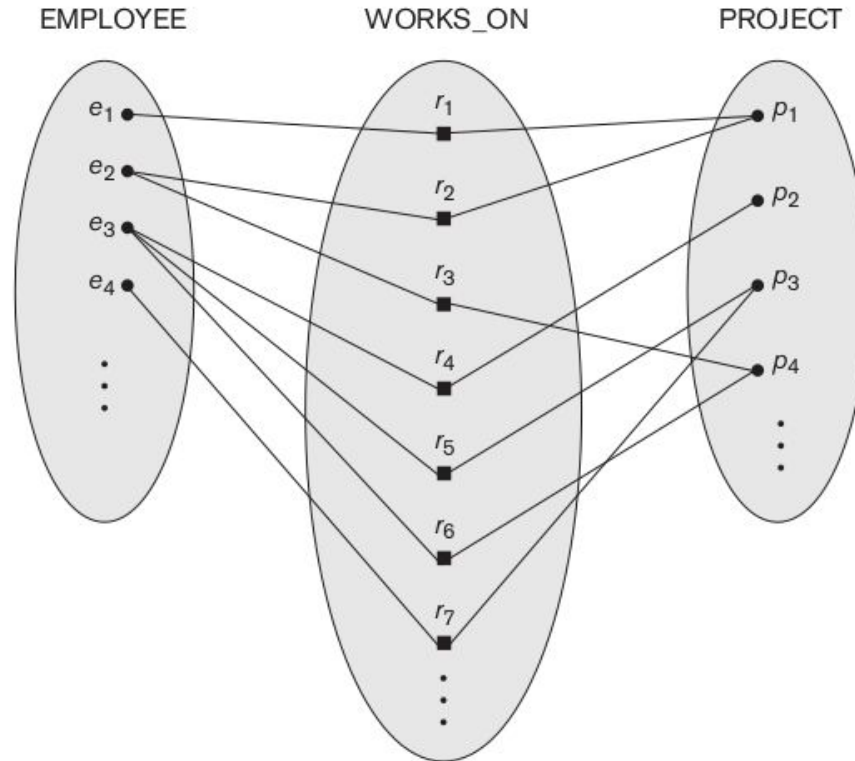


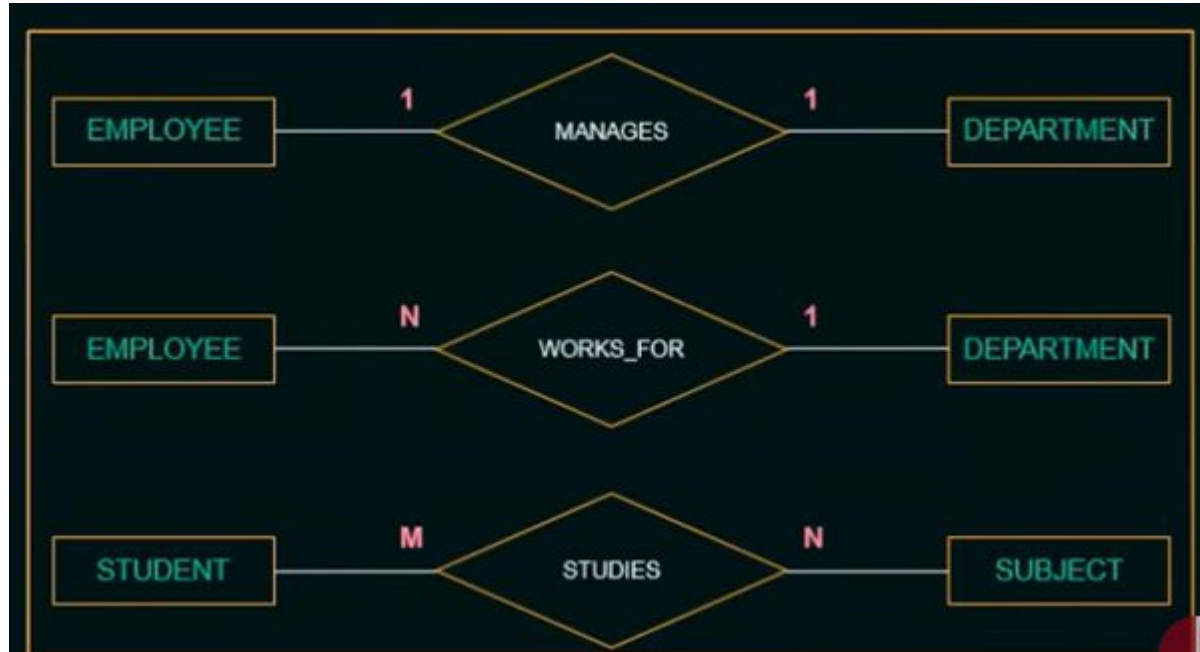
Many-to-one – When more than one instance of entity is associated with the relationship, it is marked as 'N:1'.

For example, in the WORKS_FOR binary relationship type, EMPLOYEE:DEPARTMENT is of cardinality ratio N:1, meaning that many number of employees work for one department.



Many-to-Many The relationship type WORKS_ON is of cardinality ratio M:N, an employee can work on several projects and in a project there might be several employees.





Participation Constraints and Existence Dependencies

- The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type.
- The constraint specifies the minimum number of relationship instances that each entity can participate in - **Minimum cardinality constraint**.
- There are two types of participation constraints:
 - Total Participation.
 - Partial Participation.



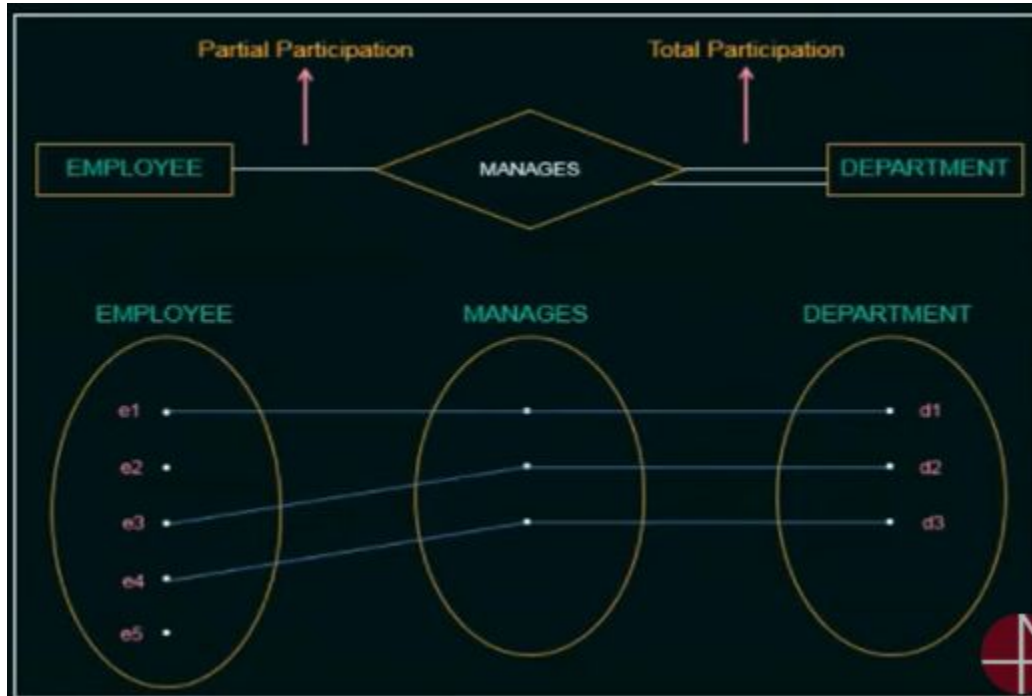
Total Participation

- Total Participation is when each entity in the entity set occurs in at least one relationship in that relationship set.
- If a company policy states that every employee must work for a department, then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance.
- Thus, the participation of EMPLOYEE in WORKS_FOR is called total participation, meaning that every entity in the total set of employee entities must be related to a department entity via WORKS_FOR.
- It is also sometimes called an **existence dependency**.
- Total participation is represented by a **double line**, going from the relationship to the dependent entity.

Partial Participation

- If only a part of the set of entities participate in a relationship, then it is called **partial participation**.
- Using the Company example, every employee will not be a manager of a department, so the participation of an employee in the “Manages” relationship is partial.
- Partial participation is represented by a **single line**.
- We do not expect every employee to manage a department, so the participation of EMPLOYEE in the MANAGES relationship type is partial, meaning that some or part of the set of employee entities are related to some department entity via MANAGES, but not necessarily all.





Attributes of Relationship Types

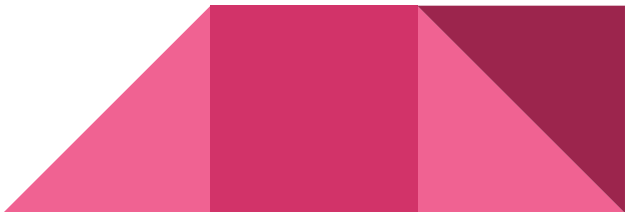
- Relationships can have attributes similar to entity types.
 - For example, in the relationship Works_On, between the Employee entity and the Department entity we would like to **keep track of the number of hours** an employee works on a project. Therefore we can include Number of Hours as an attribute of the relationship.
 - Another example is for the “manages” relationship between employee and department; we can add **Start Date** as an attribute of the Manages relationship.
 - *For some relationships (1:1, or 1: N), the attribute can be placed on one of the participating entity types. For example the “Manages” relationship is 1:1; StartDate can either be migrated to Employee or Department.*
- 

Attributes of Relationship Types

- Attributes of 1:1 or 1:N relationship types can be migrated to one of the participating entity types.
 - In 1:1 relationship type, attributes can be migrated to either of the entity types.
 - In 1:N or N:1 relationship type, attributes are migrated only to the entity type on the N-side of the relationship.

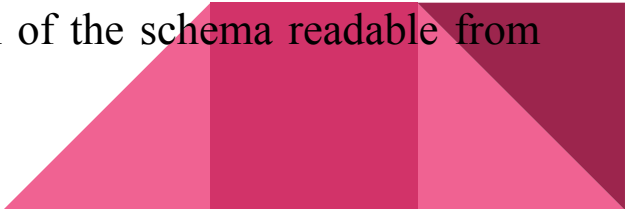
- In M:N relationship type, some attributes can be determined by a combination of participating entities.



- Consider the entity type DEPENDENT, related to EMPLOYEE, the attributes of DEPENDENT are Name, Birth_date, Sex, and Relationship (to the employee). The dependents are identified as distinct entities only after determining the particular employee entity to which each dependent is related.
 - In ER diagrams, both a weak entity type and its identifying relationship are distinguished by surrounding their **boxes and diamonds with double lines**. The partial key attribute is underlined with a **dashed or dotted line**.
- 

ER Diagrams, Naming Conventions, and Design Issues

Proper Naming of Schema Constructs

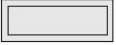
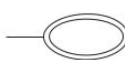
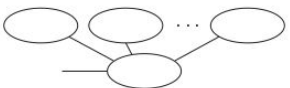
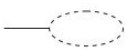
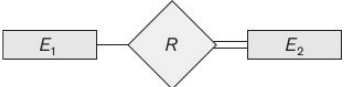
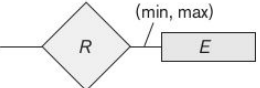
- Choose names that convey the meanings attached to the different constructs in the schema
 - Use singular names for entity types, rather than plural ones
 - Use the convention that **entity type and relationship type names are in uppercase letters, attribute names have their initial letter capitalized, and role names are in lowercase letters**
 - Nouns appearing in the narrative tend to give rise to entity type names, and the verbs tend to indicate names of relationship types
 - Choosing binary relationship names to make the ER diagram of the schema readable from left to right and from top to bottom
- 

Design Choices for ER Conceptual Design

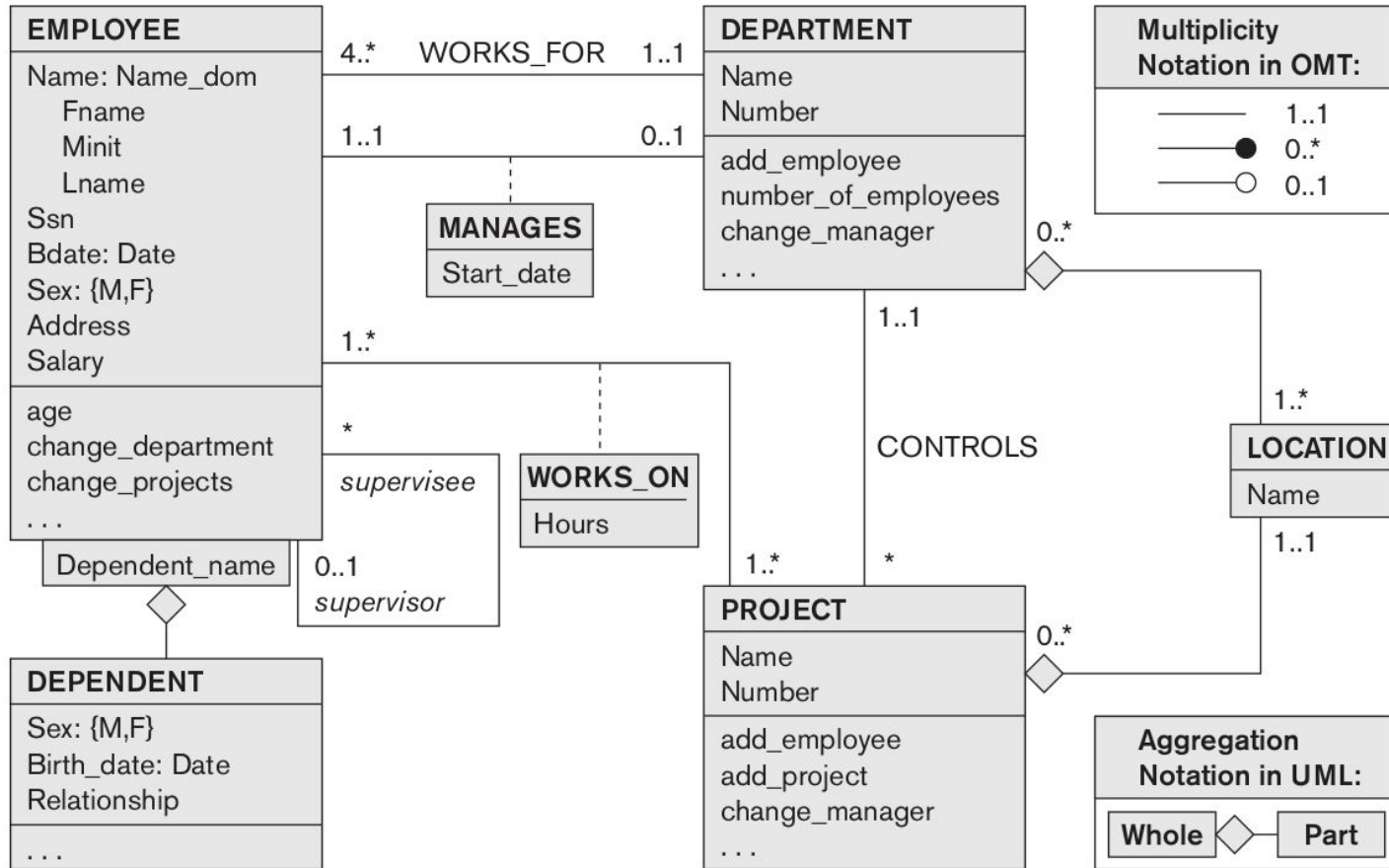
Schema design process should be considered an iterative refinement process, where an initial design is created and then iteratively refined until the most suitable design is reached. Some of the refinements that are often used include the following:

- A concept may be first modeled as an attribute and then refined into a relationship because it is determined that the attribute is a reference to another entity type
- Similarly, an attribute that exists in several entity types may be elevated or promoted to an independent entity type. For example, suppose that each of several entity types in a UNIVERSITY database, such as STUDENT, INSTRUCTOR, and COURSE, has an attribute Department in the initial design; the designer may then choose to create an entity type DEPARTMENT with a single attribute Dept_name and relate it to the three entity types (STUDENT, INSTRUCTOR, and COURSE) via appropriate relationships
- An inverse refinement to the previous case may be applied—for example, if an entity type DEPARTMENT exists in the initial design with a single attribute Dept_name and is related to only one other entity type, STUDENT. In this case, DEPARTMENT may be reduced or demoted to an attribute of STUDENT



Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1:N for $E_1 : E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

UML - Unified Modelling Language



ER Diagram	UML Class Diagram
Entity Types	Classes
Entity	Objects
Operations not specified	Class name, Attributes, Operations
Data Type	Domain name as :
Composite attribute	Structured domain
Multivalued attribute	Separate class
Relationship Types	Associations
Relationship instances	Links
Binary relationship type	Binary association
Relationship attribute	Link attribute

(min,max(*)) notation	Multiplicities
-	Unidirectional, bidirectional
Weak entities	Qualified association
Partial Key attribute	Discriminator



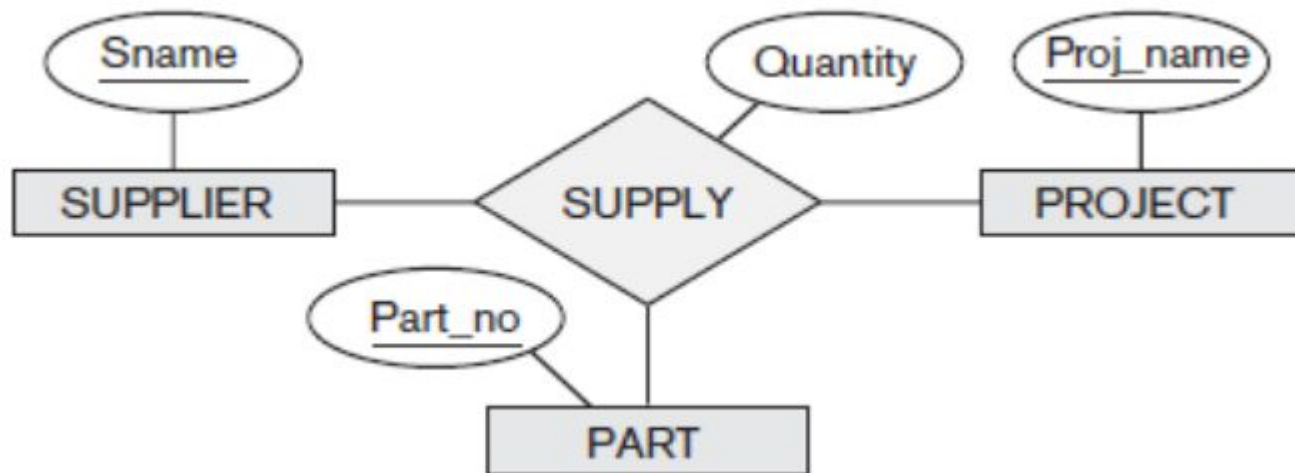
Relationship Types of Degree Higher than Two

- Relationship types of degree 2 are called binary
- Relationship types of degree 3 are called ternary and of degree n are called n -ary
- An n -ary relationship is not equivalent to n binary relationships
- In general, 3 binary relationships can represent different information than a single ternary relationship
- *The relationship set of SUPPLY is a set of relationship instances (s, j, p) , where s is a SUPPLIER who is currently supplying a PART p to a PROJECT j .*

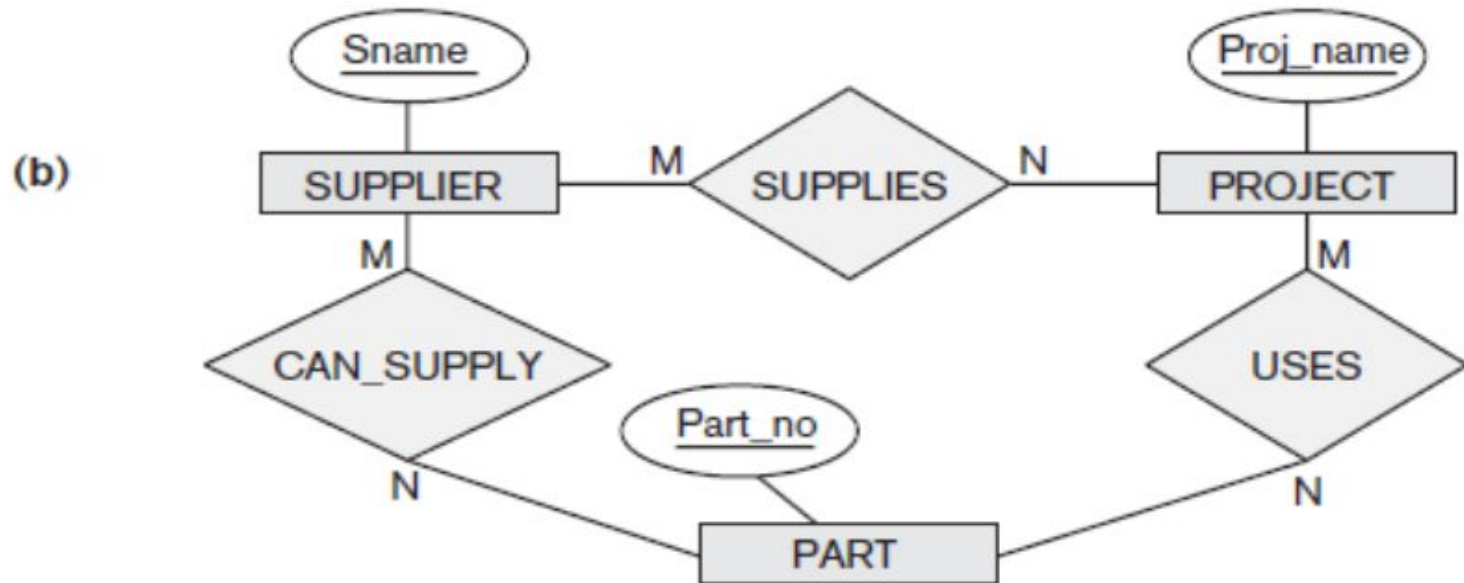


In Figure 7.17(a), The relationship set of SUPPLY is a set of relationship instances (s, j, p) , where s is a SUPPLIER who is currently supplying a PART p to a PROJECT j .

(a)



- The Figure shows an ER diagram for three binary relationship types *CAN_SUPPLY*, *USES*, and *SUPPLIES*. In general, a ternary relationship type represents different information than do three binary relationship types. Consider the three binary relationship types *CAN_SUPPLY*, *USES*, and *SUPPLIES*. Suppose that *CAN_SUPPLY*, between *SUPPLIER* and *PART*, includes an instance (s, p) whenever supplier s can supply part p (to any project); *USES*, between *PROJECT* and *PART*, includes an instance (j, p) whenever project j uses part p ; and *SUPPLIES*, between *SUPPLIER* and *PROJECT*, includes an instance (s, j) whenever supplier s supplies.



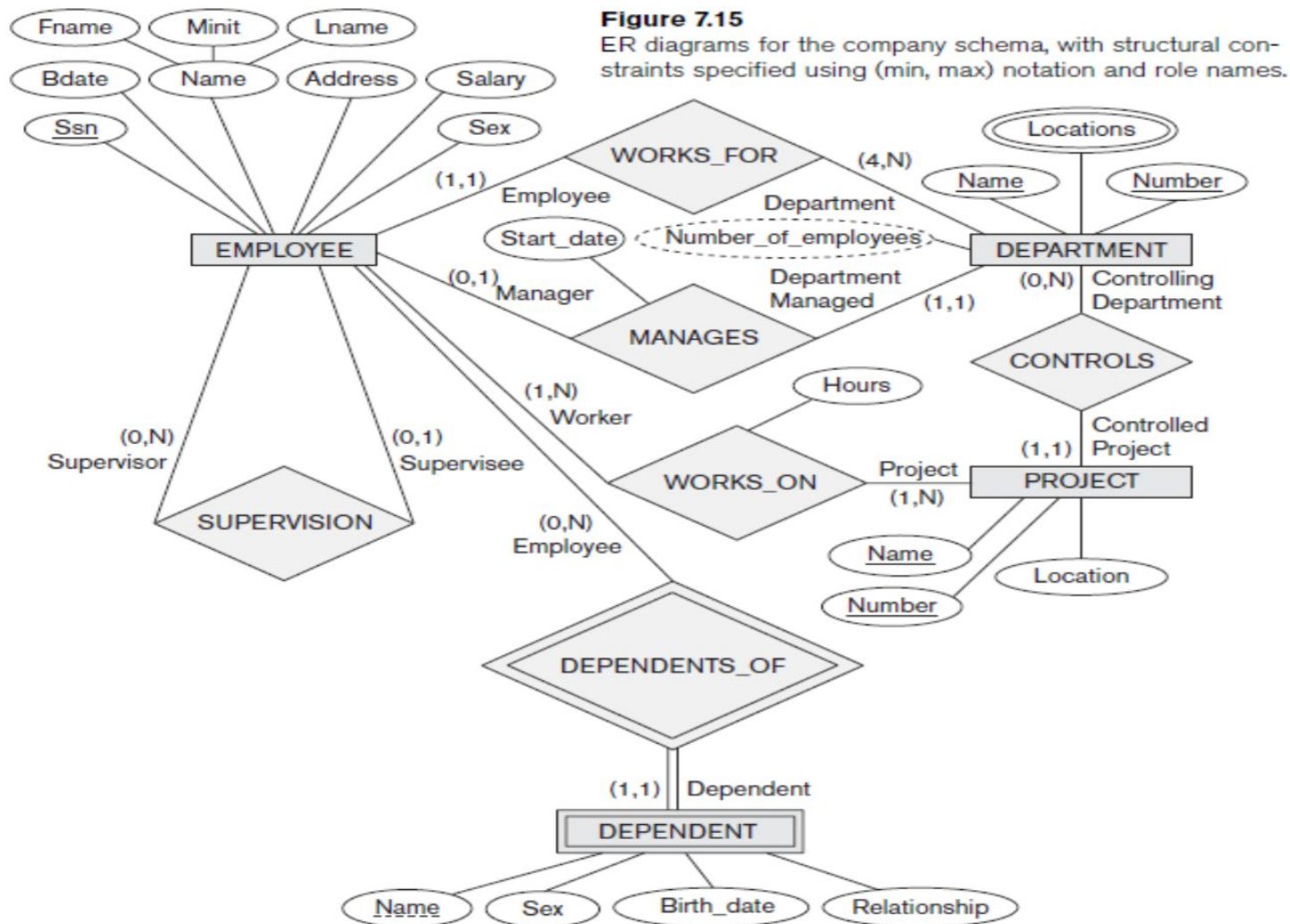
- The designer must decide whether a relationship of degree n or should be broken down into smaller degrees based on the semantics or meaning of the particular situation



Refining the ER Design for the COMPANY Database

- There is one alternative ER notation for specifying structural constraints on relationships, which **replaces** the cardinality ratio (1:1, 1:N, M:N) and single/double line notation for participation constraints.
 - This notation involves associating a pair of integer numbers (**min, max**) with each participation of an entity type E in a relationship type R, where $0 \leq \text{min} \leq \text{max}$ and **max** ≥ 1 .
 - The numbers mean that for each entity e in E, e must participate in at least min and at most max relationship instances in R at any point in time.
 - In this method, **min = 0 implies partial participation**, whereas **min > 0 implies total participation**. Figure displays the COMPANY database schema using the (min, max) notation.
- 





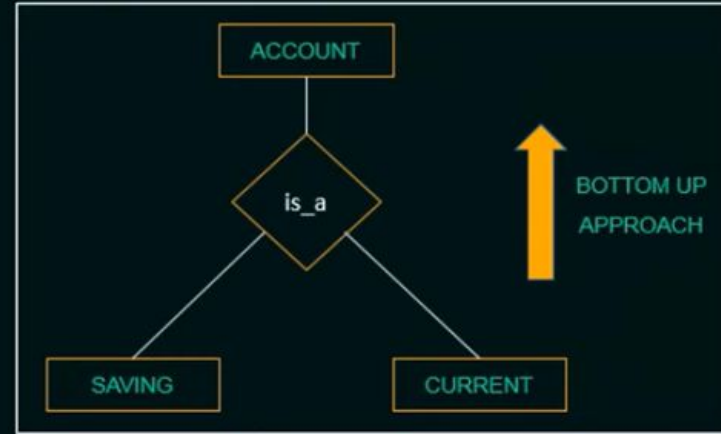
QUESTIONS

1. With neat diagram, describe different phases of database design.
 2. Define the following with respect to ER diagram:
 - a. Entity
 - b. Relationship
 - c. Attributes and types
 - d. Weak entity
 - e. Identifying relationship type
 3. List and explain different types of constraints on relationship types.
 4. Write the notations of ER diagrams.
 5. Write ER diagram for COMPANY database, MOVIE database, AIRLINE reservation database, library management database and movie database by considering miniworld requirements.
 6. Describe specialization and generalization with example.
- 

❖ Enhanced ER Model

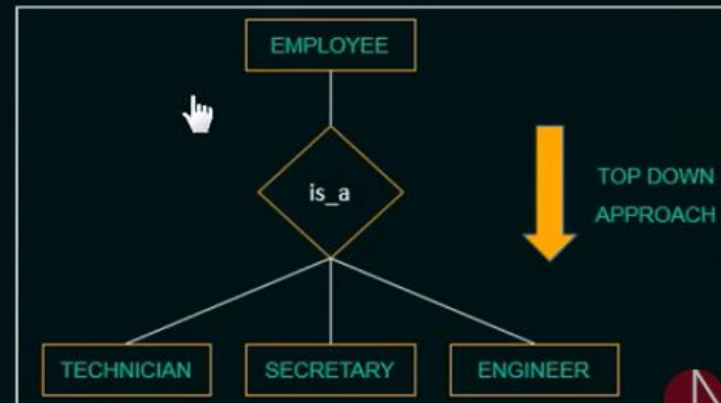
★ Generalization:

- **Bottom-up approach** where two lower level entities combine to form a higher level entity.



★ Specialization:

- **Top-down approach** where it defines set of subclasses of an entity type.



- **Specialization** is the process of defining a set of subclasses of an entity type; this entity type is called the **superclass** of the specialization.
- The set of subclasses that forms a specialization is defined on the basis of some distinguishing characteristic of the entities in the superclass

For example, the set of subclasses {SECRETARY, ENGINEER, TECHNICIAN} is a specialization of the superclass EMPLOYEE that distinguishes among employee entities based on the job type of each employee entity.

- **Figure 8.1 Represents a specialization diagrammatically in an EER diagram.** The subclasses that define a specialization are attached by lines to a **circle** that represents the specialization, which is connected in turn to the superclass. The subset symbol on each line connecting a subclass to the circle indicates the direction of the superclass/subclass relationship.

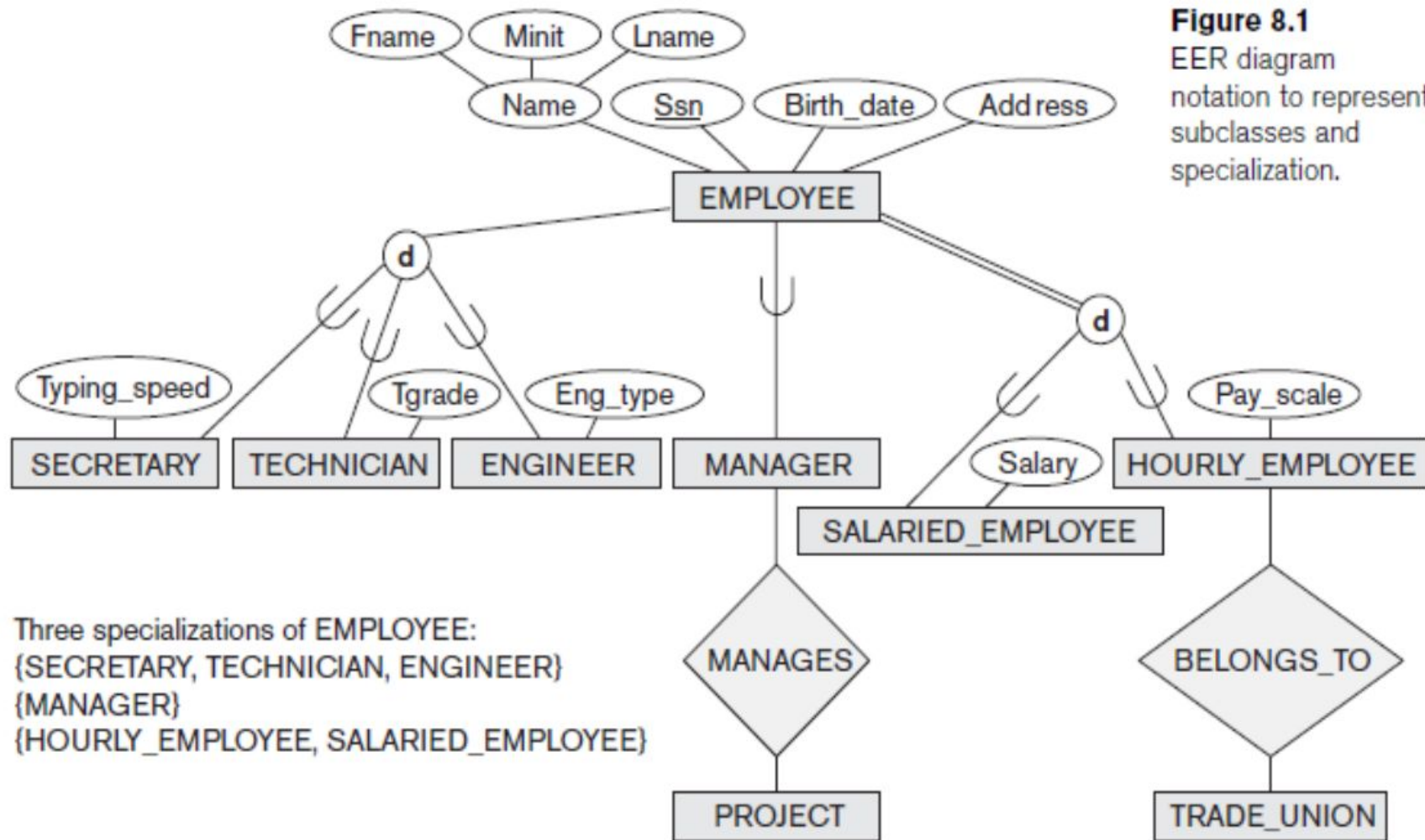


Figure 8.1
EER diagram notation to represent subclasses and specialization.

- Attributes that apply only to entities of a particular subclass such as Typing Speed of SECRETARY are attached to the rectangle representing that subclass. These are called **specific attributes** of the subclass.

Figure 8.2 shows entity instances that belong to subclasses of the {SECRETARY, ENGINEER, TECHNICIAN} specialization.

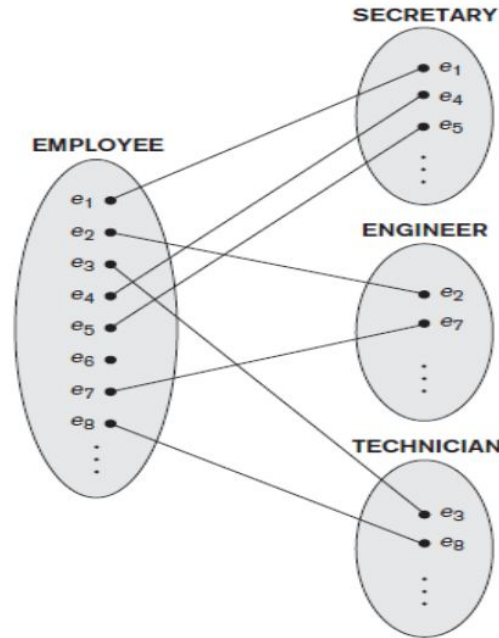


Figure 8.2
Instances of a specialization.

- There are two main reasons for including class/subclass relationships and specializations in a data model.

- The first is that certain attributes may apply to some but not all entities of the superclass. For example, in *Figure 8.1 the SECRETARY subclass has the specific attribute Typing_speed, whereas the ENGINEER subclass has the specific attribute Eng_type, but SECRETARY and ENGINEER share their other inherited attributes from the EMPLOYEE entity type.*
- The second reason for using subclasses is that some relationship types may be participated in only by entities that are members of the subclass. For example, if only HOURLY_EMPLOYEES can belong to a trade union, we can represent that fact by creating the subclass HOURLY_EMPLOYEE of EMPLOYEE and relating the subclass to an entity type TRADE_UNION via the BELONGS_TO relationship type,
- In summary, the specialization process allows us to do the following:
 - Define a set of subclasses of an entity type
 - Establish additional specific attributes with each subclass
 - Establish additional specific relationship types between each subclass and other entity types or other subclasses

Generalization



- The reverse process of abstraction suppresses the differences among entity types, identify their common features, and **generalize** them into a single **superclass** of which the original entity types are special **subclasses**.

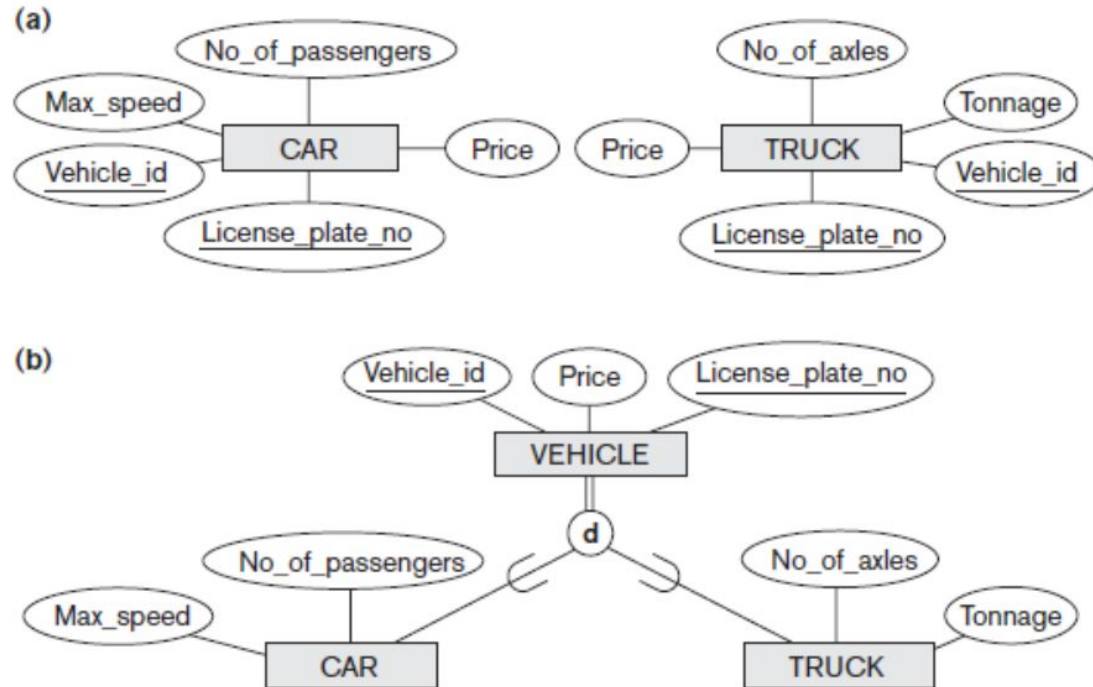


Figure 8.3

Generalization. (a) Two entity types, CAR and TRUCK. (b) Generalizing CAR and TRUCK into the superclass VEHICLE.

For example, consider the entity types CAR and TRUCK. Because they have several common attributes, they can be generalized into the entity type VEHICLE, as shown in Figure 8.3(b). Both CAR and TRUCK are now subclasses of the generalized superclass VEHICLE.

- An arrow pointing to the generalized superclass represents a generalization, whereas arrows pointing to the specialized subclasses represent a specialization.



THANK YOU

